

דוח תרגיל 3

מעבדה בבינה מלאכותית-

שם המגיש: אלמוג אמויאל 318679958

- הוראות הרצה:

גרסת פייתון: Python 3.13

ספריות סטנדרטיות:

השתמשתי לקוד בספריות המובנות: sys, os, re, copy, random, time וכן גם ב
heapq, deque. ספריות אלו משמשות בשני החלקים לאקראיות וקביעת גרעיני
ריצה, מדידת זמנים, פעולות מערכת הפעלה, חילוץ נתונים משמות קבצים, ניהול
תור עדיפויות ב-A* ושכפול של מבני נתונים ומסלולים ללא חשש.

ספריות לא סטנדרטיות:

- Numpy משמשת בחלק א' לחישובים, ובעיקר לחישוב מטריצת המרחקים ב
CVRPProblem. וכן משמשת גם לחישובי ממוצע וסטיית תקן וכן לעדכון
לעבודה עם פונקציית ACKLEY.
- Matplotlib משמשת בחלק א ליצירת שני הפלוטים שמוחזרים (אחד מהם נדרש
בתרגיל והשני הוא לעזר) שהם: מפת המסלולים וגרף השוואה בין
האלגוריתמים.

הרצה:

חלק א': CVRP

Python CVRP.py

קבצי ה vrp של המופעים חייבים להיות בתיקייה.

חלק ב': Rush hour GP

python RH_GP.py rh.txt

הקובץ rh.txt מהתרגיל הראשון, מכיל את חידות ה RH.

וכן יהיו גם קבצי exe מוכנים להרצה וקובץ readme מפורט.

חלק א' - CVRP:

1. מבוא:

בעיית ה CVRP מהווה כאן הרחבה לבעיית TSP מהתרגיל הקודם, כשבבעיה הנוכחית קיים צי של רכבים בעלי קיבולת מוגבלת שיוצאים ממחסן מרכזי וזאת במטרה לספק סחורה לאוסף לקוחות עם דרישה משתנה. המטרה כאן הינה למצוא סט של מסלולים, אחד לכל רכב, שמתחיל ומסתיים במחסן, כך שכל לקוח יקבל שירות רק פעם אחת, אף רכב לא יחרוג מהקיבולת שלו, והמרחק הכולל שיעברו יהיה מינימלי.

מימשתי והשוויתי שבעה אלגוריתמי חיפוש שונים על פני מרחב של בעיות CVRP שונות מהמאגר הנתון, זאת מבעיות קטנות ועד בעיות גדולות ומסובכות יותר. וכבדיקת שפיות, בחנתי את האלגוריתמים על פונקציית Ackley לאימות של היכולת שלהם למצוא מינימום גלובלי במרחב.

2. ייצוג הבעיה:

הפתרון הוא במבנה מונחה עצמים, והמחלקות המרכזיות במימוש הן:

א. Node:

מחלקה המייצגת מיקום בודד על המפה של מחסן או לקוח וכוללת קואורדינטות x,y שלו ואת הקיבולת שלו.

ב. CVRPPProblem:

מחלקה זו שומרת את הנתונים הסטטיים של הבעיה. החשוב כאן הוא החישוב מראש של מטריצת המרחקים המלאה עם הפונקציה `_calculate_distance_matrix` כך שמבטיח לנו סיבוכיות $O(1)$ במקום חישוב שורש ריבועי מחדש בכל פעם, וזה יעיל למופעים הגדולים.

```
def __init__(self, name, num_vehicles, capacity, nodes):
    self.name = name
    self.num_vehicles = num_vehicles
    self.capacity = capacity
    self.nodes = nodes
    # 0 is the warehouse
    self.depot = nodes[0]
    self.customers = nodes[1:]

    self.num_nodes = len(nodes)
    self.distance_matrix = np.zeros((self.num_nodes, self.num_nodes))
    self._calculate_distance_matrix()

def _calculate_distance_matrix(self):
    """calculate euclidean distance between all nodes once"""
    for i in range(self.num_nodes):
        for j in range(self.num_nodes):
            if i != j:
                node_i = self.nodes[i]
                node_j = self.nodes[j]
                dist = np.sqrt((node_i.x - node_j.x)**2 + (node_i.y - node_j.y)**2)
                self.distance_matrix[i][j] = dist
```

ג. CVRPSolution:

מייצגת פתרון חוקי, ומנהלת מצב דינאמי של צי הרכבים. ייצוג הפתרון הוא במבנה נתונים של רשימה של רשימות ובחור מכמה שיקולים:

- כל אינדקס מייצג רכב, והרשימה הפנימית את סדר הביקור אצל הלקוחות. המחסן לא נשמר פיזית במסלולים, אבל נוסף בחישובי העלות.
- גם כך אפשרי למטא יוריסטיקות לבצע שינויים כמו swap, relocate, מהר, עם ניצול המבנה של הרשימה.
- מחשבים בפונקציית חישוב העלות לכל מסלול בנפרד.

```
def __init__(self, problem):
    self.problem = problem
    # a list of lists- sub list is a route containing node objects
    self.routes = [[] for _ in range(problem.num_vehicles)]
    self.total_cost = 0.0

def calculate_cost(self):
    """
    calculates the total distance of all routes, depot-first customer-next customers-depot
    """
    total = 0.0
    for route in self.routes:
        #skip empty routes
        if not route:
            continue
        dist = self.problem.get_distance(self.problem.depot.id, route[0].id)
        for i in range(len(route) - 1):
            dist += self.problem.get_distance(route[i].id, route[i+1].id)
        dist += self.problem.get_distance(route[-1].id, self.problem.depot.id)
        total += dist
    self.total_cost = total
    return total
```

3. האלגוריתמים:

א. Multistage heuristic:

יוריסטיקה זו מהווה בסיס ופתרון התחלתי לשאר האלגוריתמים. ופועלת בשני שלבים:

- מיון הלקוחות, כמו knapsack, שבו הלקוחות ממוינים לפי מרחק מהמחסן, מהרחוק לקרוב, כך שהקשים והרחוקים יותר ישובצו קודם.
- שיבוץ כל לקוח למסלול קיים או פתיחת מסלול חדש בעזרת nearest neighbor אשר משמש כבסיס להשוואה ולנקודת התחלה נוחה לאלגוריתמים הבאים, שעליהם נרחיב כאן.

וכדי להבטיח שלמות של הפתרון גם במופעים עם עומס רב בהם נגמר המקום בכל המשאיות לפני ששובצו כל הלקוחות, הוספתי מנגנון גיבוי שלפיו ללקוח שאין בשבילו קיבולת פנויה ישובץ למשאית הפחות עמוסה, ומבטיח בכך כיסוי מלא של כל הלקוחות, בהסתמך על כך שמימוש האלגוריתמים הבאים יתקן את חריגת הקיבולת.

סיבוכיות: שלב מיון הלקוחות מתבצע בסיבוכיות $O(n \log n)$, לאחר מכן, לולאת השיבוץ החמדנית עוברת על כל n הלקוחות, ולכל לקוח מתבצעת בדיקת שיבוץ ומגבלת קיבולת מול כל אחד מ k הרכבים בצי $O(nk)$, סה"כ $O(n \log n + nk)$.

```
def greedy_multi_stage_heuristic(problem):

    solution = CVRPSolution(problem)
    unassigned = sorted(problem.customers, key=lambda c: problem.get_distance(problem.depot.id, c.id), reverse=True)

    vehicle_loads = [0] * problem.num_vehicles

    for customer in unassigned:
        best_vehicle = -1
        best_insert_cost = float('inf')

        for v in range(problem.num_vehicles):
            #check capacity constraint
            if vehicle_loads[v] + customer.demand <= problem.capacity:
                if not solution.routes[v]:
                    #cost of opening a new route
                    cost = problem.get_distance(problem.depot.id, customer.id) * 2
                else:
                    #cost of adding to the end of an existing route
                    last_customer = solution.routes[v][-1]
                    cost = problem.get_distance(last_customer.id, customer.id)

                if cost < best_insert_cost:
                    best_insert_cost = cost
                    best_vehicle = v

        if best_vehicle != -1:
            solution.routes[best_vehicle].append(customer)
            vehicle_loads[best_vehicle] += customer.demand
        else:
            #no vehicle has room- put in least loaded vehicle, meta heuristic repairs overloads
            least_loaded = min(range(problem.num_vehicles), key=lambda v: vehicle_loads[v])
            solution.routes[least_loaded].append(customer)
            vehicle_loads[least_loaded] += customer.demand

    solution.calculate_cost()
    return solution
```

ב. Tabu search:

חיפוש מקומי אשר סורק את מרחב הפתרונות עם אופרטור relocate של העברת לקוח ממסלול למסלול. וכדי להימנע ממעגלים ולצאת ממינימום מקומי משתמש האלגוריתם ברשימת טאבו שאוסרת על חזרה למהלכים שכבר בצענו למשך מספר קצוב של איטרציות. ולפי שלמדנו שלבתי במנגנון גם Aspiration criterion שמאפשר לדרוס את איסור הטאבו במקרה בו המהלך החסום כן יוכל להביא לפתרון הגלובלי הטוב ביותר שמצאנו עד עכשיו.

יישום strategic oscillation הוטמע בפונקציה זו ב SA וב ALNS, פונקציות יצירת השכנים לא חוסמות העברת לקוח למשאית מלאה, אלא מעריכות את כל השכנים דרך פונקציית calculate_fitness המכילה את הקנס, ומאפשר החלטות מושכלות גם במרחב לא חוקי.

סיבוכיות: בכל איטרציה מתוך I האיטרציות, האלגוריתם דוגם סביבת שכונה של m שכנים מועמדים ($m=30$ אצלי). עבור כל שכן מועמד מתבצע חישוב fitness ובדיקה מול רשימת הטאבו בסיבוכיות $O(n)$, שמניב עלות של $O(I*m*n)$.

ג. Simulated annealing:

אלגוריתם סטוכסטי שמתחיל בטמפרטורה גבוהה שמאפשרת קבלת מהלכים מרעים בהסתברות גבוהה, ומדלג כך מעל מינימום מקומי במרחב של CVRP. ככל שמתקדם הטמפרטורה דועכת וההסתברות לקבל מהלכים רעים יורדת מאוד לפי קריטריון מטרופוליס $P = e^{-\Delta E/T}$ זאת עד שהאלגוריתם יתכנס בהדרגה למצב של Hill climbing לביצוע Fine tuning.

סיבוכיות: בכל אחת מ I האיטרציות, האלגוריתם מייצר שכן בודד עם אופרטור relocate שמבצע שינוי מקומי, והערכת הפיטנס שלו (שמרתי מטריצת מרחקים) מתבצעת בסיבוכיות המוגבלת לאורך המסלול, סך הכל $O(I*n)$.

ד. Ant colony optimization:

מטא יוריסטיקה מבוססת אוכלוסייה. בכל איטרציה, מושבת נמלים בונה פתרונות מאפס, ההסתברות של נמלה לבחור בלקוח הבא מחושבת כשילוב שני גורמים:

- פרומון המייצג למידה גלובלית מפתרונות עבר מוצלחים.
- מידע יוריסטי מקומי $\eta = 1/\text{dist}$, שמעודד בחירת לקוחות קרובים.

בסוף של כל דור מבוצע מנגנון Evaporation קבוע שמונע התכנסות מוקדמת, והנמלים המוצלחות מפקידות פרומון נוסף לחיזוק המסלולים האידיאליים.

במופעים חריגים, כמו ב $X-n101$, נמלה עלולה להגיע למצב בו נותרו לקוחות אבל אין רכב פנוי שיכול להכיל אותם. ובכדי למנוע קריסה או לולאה אינסופית הוספתי מנגנון של השלמת מסלול שמחלץ את הלקוחות הנותרים למסלולים וירטואליים חדשים, כך כל נמלה תחזיר תמיד פתרון מלא שמקבל קנס כמו שצריך, ועדכון הפרומונים לא נפגע.

סיבוכיות: בכל דור מתוך I הדורות, כל אחת מ a הנמלים בונה פתרון שלם, ובכל שלב בבניית המסלול, הנמלה מחשבת הסתברויות ומעריכה את כלל הלקוחות שעוד לא בוקרו, תהליך הדורש $O(n)$ צעדים ושכל אחת מהם נסרקים עד $O(n)$ לקוחות, מכאן שסך הכל $O(I*a*n^2)$.

מבין כל מימושי הקוד הארוכים בחרתי להדביק רק את המימוש הזה, כדי לא להעמיס על הדוח.

```
def aco_search(problem, num_ants=10, max_iter=20):  
  
    # initialize pheromone matrix  
    pheromones = np.full((problem.num_nodes, problem.num_nodes), 0.1)  
    best_sol = None  
  
    for _ in range(max_iter):  
        solutions = []  
        for _ in range(num_ants):  
            sol = CVRPSolution(problem)  
            unvisited = set(node.id for node in problem.customers)  
  
            #build routes for each vehicle  
            for v in range(problem.num_vehicles):  
                if not unvisited: break  
                curr_node_id = problem.depot.id  
                curr_cap = 0  
  
                while unvisited:  
                    probs = {}  
                    #calculate probabilities for valid next steps  
                    for target_id in unvisited:  
                        target_node = problem.nodes[target_id]  
                        if curr_cap + target_node.demand <= problem.capacity:  
                            dist = problem.get_distance(curr_node_id, target_id)  
                            # heuristic- 1/distance  
                            eta = 1.0 / dist if dist > 0 else 0  
                            tau = pheromones[curr_node_id][target_id]  
                            probs[target_id] = (tau ** 1.0) * (eta ** 2.0)  
  
                    if not probs: break  
  
                    #selection based on probabilities  
                    total_prob = sum(probs.values())  
                    rand_val = random.uniform(0, total_prob)  
                    cumulative = 0.0  
                    next_node_id = None  
                    for tid, p in probs.items():  
                        cumulative += p  
                        if cumulative >= rand_val:  
                            next_node_id = tid  
                            break  
  
                sol.add_route(curr_node_id, next_node_id)  
                curr_node_id = next_node_id  
                curr_cap += target_node.demand  
                unvisited.remove(target_id)  
  
            solutions.append(sol)  
  
        best_sol = min(solutions, key=lambda sol: sol.get_cost())  
    return best_sol
```

```

        #move the ant
        sol.routes[v].append(problem.nodes[next_node_id])
        unvisited.remove(next_node_id)
        curr_cap += problem.nodes[next_node_id].demand
        curr_node_id = next_node_id

# on tight instance the num_vehicles loop above can end with unvisited customers,
# so before I noticed, the customers were dropped.
# the ants solution was not good- best_sol stayed none, and returned inf, which raised RuntimeWarning
# so now I put the remaining customers into extra routes so every ant returns a full solution
while unvisited:
    extra_route = []
    extra_cap = 0
    for cid in sorted(unvisited):
        if extra_cap + problem.nodes[cid].demand <= problem.capacity:
            extra_route.append(problem.nodes[cid])
            extra_cap += problem.nodes[cid].demand
    for node in extra_route:
        unvisited.discard(node.id)
    if not extra_route:
        #single customer whose demand alone exceeds capacity- give him his own route to avoid infinite loop
        cid = next(iter(unvisited))
        extra_route.append(problem.nodes[cid])
        unvisited.discard(cid)
    sol.routes.append(extra_route)

sol.calculate_cost()
solutions.append(sol)

#keep track of global best valid solution
if not unvisited:
    if best_sol is None or sol.total_cost < best_sol.total_cost:
        best_sol = sol

# pdate pheromone- evaporation & deposit
#evaporation rate rho=0.1
pheromones *= 0.9
for s in solutions:
    if s.total_cost == 0: continue
    # better routes leave more pheromone
    deposit = 100.0 / s.total_cost
    for r in s.routes:
        if not r: continue
        pheromones[problem.depot.id][r[0].id] += deposit
        for i in range(len(r) - 1):
            pheromones[r[i].id][r[i + 1].id] += deposit
        pheromones[r[-1].id][problem.depot.id] += deposit

return best_sol

```

ה. Adaptive large neighborhood search:

ALNS - בשונה מחיפוש מקומי רגיל שמשנה מיקום של לקוח אחד בכל פעם, ALNS מבצע ארגון מחדש עמוק לפתרון. בכל איטרציה, אופרטור הרס שולף באקראי תת קבוצה גדולה של לקוחות, ולאחריו אופרטור תיקון חמדני משבץ אותם מחדש בעלויות המינימליות האפשריות. כך שהוא מדלג מעל עמקים גדולים במרחב החיפוש, והוא האלגוריתם החזק והדומיננטי עבור המופעים הגדולים והבינוניים.

הוספתי גם לאופרטור התיקון קנס רך, לפיו האלגוריתם יעדיך תמיד לשבץ לקוח במקום שבו יש קיבולת פנויה ויקנוס שיבוץ שגוי, אבל לא יחסום חריגה לגמרי, ובכך נשמר החיפוש מפיזור של כולם למשאית אחת, אבל מאפשר חילוץ gridlock.

סיבוכיות: בכל איטרציה אופרטור ההרס שולך תת קבוצה של עד $O(n)$ לקוחות, שלב התיקון משבץ אותם יחד אחד אחרי השני, כאשר עבור כל לקוח שנשאר מחוץ למסלול, האלגוריתם סורק את כל מיקומי ההכנסה האפשריים לאורך כל המסלולים - $O(n)$ מיקומים, ומכאן בכולל $O(n^2)$.

ו. Branch & bound with LDS

אלגוריתם חיפוש עצים- עץ החיפוש המלא של CVRP גדל מעריכית, לכן שלבתי את שיטת LDS, כך שהאלגוריתם בוטח ביוריסטיקה של מיון קודקודי הבנים לפי מרחק, אבל מאפשר לסייר בענפים פחות סבירים עד מספר מרבי שלי סטיות שמוגדר מראש. העץ החלקי הזה מבטיח סריקה חכמה ויסודית של מרחב החיפוש המקומי, אבל בשל הסיבוכיות הגדולה שלו הגבלתי אותו לשימוש במופעים קטנים של עד 31 לקוחות.

האלגוריתם מיישם גיזום עם בניית העץ המקומי, פוסל מראש בנים החורגים מהקיבולת ומחשב את העלות תמיד, כולל גם את החזרה למחסן, בכל צעד. כך מובטח שהחסם מדויק ומונע פיתוח ענפים לא קשורים.

סיבוכיות: עץ החיפוש של CVRP גדל מעריכית, ובשילוב LDS שמרסנת את הסיבוכיות על ידי הגבלה של מספר הסטיות המותרות מהיוריסטיקה החמדנית, אך לו סיבוכיות גבוהה המגבילה את השימוש בו למופעים קטנים של עד 31 לקוחות, מכאן $O(k^n)$, כאשר k מספר הרכבים בצי.

```
def branch_and_bound_lds(problem, max_discrepancies=1):
    """
    branch and bound/LDS: Sequential route building-
    B&B explodes on CVRP, but LDS gives an approximation by exploring a limited tree
    """
    solution = CVRPSolution(problem)
    unvisited = set(problem.customers)

    for v in range(problem.num_vehicles):
        if not unvisited: break
        route, _ = lds_route_builder(
            problem, v, unvisited, current_route=[],
            current_cost=0.0, max_disc=max_discrepancies, curr_disc=0
        )
        solution.routes[v] = route
        for cust in route:
            unvisited.remove(cust)

    solution.calculate_cost()
    return solution
```

2. אלגוריתם גנטי ממטי עם מודל איים:

בהמשך לתרגיל הקודם בו השתמשנו גם באלגוריתם גנטי ממטי, כאן כדי לשמר גיוון גנטי ולמנוע קיפאון באוכלוסייה, יישמתי מודל איים, שבו מספר תת אוכלוסיות מתפתחות בנפרד במקביל, ומחליפות מהגרים טובים במבנה של טבעת פעם לכמה דורות. האלגוריתם הוא ממטי שכן משלב אופרטורים גנטיים גם הם מהתרגיל הקודם (OX, swap mutation), זאת עם שיפור מקומי המפעיל את 2-opt על כל צאצא אשר מסיר הצטלבויות בתוך המסלול. וכן בקשר לייצוג הנתונים המעבר ממסלולים נפרדים לפעולות גנטיות דרש פיתוח של מנגנון השטחה ופיצול- הכרומוזומים מושטחים לרצף אחד כדי לאפשר הפעלת OX שמונע שכפול לקוחות. ואחר כך פונקציית `ga_split_into_routes` תפצל מחדש את הלקוחות למשאיות חוקיות בצורה חמדנית על בסיס הקיבולת. סיבוכיות: בכל דור מתוך l הדורות, מנוהלת אוכלוסייה של k פרטים על פני האיים, הממטיות שמפעילה אופרטור 2-opt שמופעל על כל צאצא חדש, סורקת את כל זוגות הקשתות האפשריים במסלול להסרת הצטלבויות, והוא $O(n^2)$ לכל פרט.

4. חוקיות מול אופטימליות:

במופעים הדוקים מאוד כמו X-n101-k25 הגבלת החיפוש לפתרונות שהם רק חוקיים גרמה לgridlock במרחב החיפוש. ברגע שכל המשאיות כמעט מלאות לגמרי, שום לקוח לא יכול לעבור למשאית אחרת ללא חריגה מהקיבולת, ונגיע למינימום מקומי. יישמתי פתרון אשר למדנו בהרצאה, שלפיו הסרתי את חסימות הקיבולת מאופרטורי השכונה ב-SA, Tabu ו-ALNS והוחלפו בפונקציית קנס דינמית:

$$\text{Fitness} = \text{total_distance} + (\text{penalty_weight} * \text{total_overload})$$

עם קנס במשקל של 10,000 לכל יחידת חריגה, היישום הזה אפשר לחיפוש לבצע strategic oscillation ובכך לגלוש אל המרחב הלא חוקי כדי להעביר לקוחות בין משאיות עמוסות ולשחרר חסימות, ואז להשתמש בשיפוע של הקנס כדי להתכנס לפתרון החוקי. וכדי להבטיח שהפלט תקין שמרתי את הפתרון החוקי הטוב ביותר שנמצא.

קודם כל שלבתי מנגנון לאיחוד מסלולים עודפים, אבל לאחר שהטמעת strategic oscillation, ראיתי כי הפך למיותר.

5. יוריסטיקות שפותחו:

- א. חיסכון, בשלב הבניה יש העדפה של חיבור מסלולים לחסכון בנסיעות חזרה למחסן.
- ב. Nearest neighbor: אתחול לנקודת התחלה סבירה.
- ג. 2-opt: הסרת הצטלבויות בתוך מסלולים בודדים (באלגוריתם הממטי).
- ד. שיבוץ זול ביותר: פונקציית התיקון באלגוריתם ALNS.
- ה. אופרטורי שכונה (relocate, swap, destroy) מגדירים את המבנה של עץ החיפוש באלגוריתמים השונים.
- ו. ראות נמלים- $1/d$: ההטייה היוריסטית שמכוונת את ACO לבחירת לקוחות קרובים.
- ז. קנסות רכים: היוריסטיקה המאפשרת גמישות קיבולת זמנית.

6. בחירת פרמטרים וכיול:

- א. SA: נבחרה טמפרטורה התחלתית גבוהה 100, וקצב קירור 0.95, שמאפשר קבלת מהלכים רבים מרעים בתחילה (לצורך exploration) כדי להיחלץ ממינימום מקומי, ודעיכה הדרגתית שמביאה להתכנסות בסוף (exploitation). עבור Ackley הגדרתי גודל צעד לפי התקדמות שמאפשר לאלגוריתם לחמוק ממינימום מקומיים רבים לכיוון הגלובלי.
 - ב. ACO: משקל המידע היוריסטי β הוגדר גבוה יותר ממשקל הפרומון α זאת כדי למנוע התכנסות מוקדמת לאזור לא אופטימלי בדורות הראשונים. אתחול הפרומון המקורב נקבע לערך של 0.1 וזאת עם קצב evaporation של $\rho=0.1$, כדי למחוק בהדרגה מסלולי גרועים שעברנו, וכן הורץ עם 10 נמלים בכל דור.
 - ג. Tabu search: זיכרון הטאבו נקבע לגודל מתון של 10 איטרציות, שכן יהיה ארוך מספיק למנוע מעגליות, אבל קצר מספיק כדי שלא לשתק את חופש הפעולה של השכונה. כדי לשמור על זמן ריצה נורמלי במופעים גדולים, נדגמו בכל איטרציה 30 שכנים אקראיים מתוך סביבת relocate.
 - ד. ALNS: גודל ההרס הותאם דינאמית לגודל המופע- טווח שנע בין 2 לקוחות ועד $n/4$ מכלל הלקוחות, כך שהשינוי יהיה משמעותי במופעים גדולים מידי, אבל לא יהרוס לגמרי מופעים קטנים.
 - ה. GA island: כיול של גודל האוכלוסייה ל-40 פרטים שרצים ב-100 דורות, מספר איים, תדירות ההגירה- בטופולוגית טבעת שבה איים שכנים מחליפים מהגרים מצטיינים, ותדירות ההפעלה הממטית- אופרטור ה-2-opt, כך שיתקבל איזון ראוי בין איכות הפתרון לזמן הריצה.
 - ו. כיול משקל הקנס- strategic oscillation: משקל הקנס על חריגה מאילוץ הקיבולת בפונקציית הפיטנס נקבע לערך גבוה של 10000 לכל יחידת עומס עודפת, ומבטיח כי כל פתרון חוקי יהיה תמיד עדיף על אחד לא חוקי.
- כל האלגוריתמים הורצו תחת אותם 3 גרעינים 42,123,999. ובגלל סטוכסטיות האלגוריתמים מדווחים ערכי המינימום, הממוצע וסטיית התקן.

7. בדיקת שפיות- Ackley:

לפני פתרון הבעיה וניתוב המשאיות, נדרש היה לוודא כי האלגוריתמים מסוגלים למצוא מינימום גלובלי במרחב רציף עם מינימום/מקסימום מקומיים רבים. הבדיקה בוצעה על פונקציית Ackley במרחב של 10 מימדים, ובגבולות שנדרש במשימה. לשם כך בצעתי הכללה והתאמה של האלגוריתמים המרכזיים לחיפוש במרחב רציף, ובחנתי אותם על Ackley. החרגתי את B&B/LDS מהבדיקה, שכן מדובר בחיפוש סדרתי ובדיד שלא מיפיתי לרציפות כמו השאר.

התאמת המטא יוריסטיקות לרציפות:

- **Simulated annealing**: יישום רגיל של SA שמייצר שכנים עם הפרעות גאוסיות ודעיכת טמפרטורה שמאפשרת קבלה של מהלכים רעים בהסתברות יורדת. הראשונה אשר מימשתי בתחילה, ולאחר מכן בעקרון דומה ממשתי את השאר.

```
def simulated_annealing_ackley(d=10, initial_temp=10.0, cooling_rate=0.9995, max_iterations=100000):
    current_x = np.random.uniform(-32.768, 32.768, size=d)
    current_energy = ackley_function(current_x)

    best_x = np.copy(current_x)
    best_energy = current_energy
    temp = initial_temp

    for it in range(max_iterations):
        #step lowered linearly with search progress- exploration early, fine tuning later
        # floor of 0.001 lets it settle into basin
        frac = it / max_iterations
        step = 2.0 * (1 - frac) + 0.001
        neighbor_x = np.clip(current_x + np.random.uniform(-step, step, size=d), -32.768, 32.768)
        neighbor_energy = ackley_function(neighbor_x)

        diff = neighbor_energy - current_energy
        if diff < 0 or random.random() < np.exp(-diff / max(temp, 1e-9)):
            current_x = neighbor_x
            current_energy = neighbor_energy
            if current_energy < best_energy:
                best_x = np.copy(current_x)
                best_energy = current_energy

        temp *= cooling_rate
        if best_energy < 1e-4:
            break

    return best_x, best_energy
```

- **GA**: הותאם לGA רציף בו הצאצאים מיוצרים עם אופרטור ממוצע (Blend crossover: $(a+b)/2$) ומוטציה גאוסית שמופעלת בהסתברות קבועה על מרכיבי הוקטור.
- **Tabu search**: בכדי לאפשר טאבו במרחב רציף, יישמתי מנגנון המעגל את קואורדינטות הוקטור לשבר עשרוני אחד $\text{round}(c,1)$. ומפתח זה נשמר ברשימת הטאבו ומונע מהאלגוריתם לחזור לאותו תא מרחבי, תוך הקטנת צעד החיפוש.

- **ALNS**: אופרטור destroy מיוצג בבחירת תת קבוצה אקראית של קואורדינטות בווקטור מא ממדים, ואופרטור repair שדוגם מחדש את ערכיהן בסביבת הערך הנוכחי עם תוספת של רעש נורמלי.
- **ACO**: גם כאן התאמה של ACO לרציפות שדרושה מ Ackley. האלגוריתם ינהל ארכיון של פתרונות טובים שמהווה חלופה לטבלת הפרומון הבדידה. בכל איטרציה, נמלה דוגמת פתרון חדש מתוך התפלגות נורמלית שמרכזת בפתרון מהארכיון, וסטיית התקן שלה נגזרת מפיזור של הפתרונות בארכיון באותו ממד.
- **Greedy**: מקביל ליוריסטיקה רב שלבית של CVRP, חיפוש hill climbing שמקבל רק מהלכים משפרים, ומופעל ממספר נקודות התחלה אקראיות.

ACKLEY			
Algorithm	Best	Mean	Std
SA	0.002155	0.003238	0.000774
GA	0.123341	0.141557	0.018637
Tabu	0.072158	0.083126	0.008514
ALNS	0.328495	0.798412	0.379114
ACO	0.013985	0.017028	0.002182
Greedy	19.049143	19.277324	0.194348
(target global minimum = 0.0)			

ניתוח התוצאות ACKLEY:

- **הצלחת המטא יוריסטיקות**: אלגוריתמי ALNS, Tabu, GA, SA, ACO כולם השיגו תוצאות יחסית טובות באזור האפס, שקרובות למינימום המוחלט 0. ובכך זה מאשר את התקינות של מנגנוני החקר, הזיכרון וההתכנסות שמומשו בתרגיל.
- **הכשלון של החיפוש החמדני**: כצפוי ממה שלמדנו בקורס, האלגוריתם החמדני הציג ביצועים גרועים מאוד ונותר רחוק באופן משמעותי מהמינימום הגלובלי, זאת מכיוון שמסתמך על קבלת שיפורים בלבד, ונתפס במינימום מקומיים הרבים ב Ackley. וזה ממחיש למה אסור להסתפק בשיטות חמדניות לפתרון בעיות מורכבות כמו CVRP, ובכך מצדיק שימוש באלגוריתמים המתקדמים שמימשתו בתרגיל.

ניתוח תוצאות:

problem: P-n16-k8.vrp (customers=15, vehicles=8, capacity=35)							
Algorithm	Best	Mean	Std	Elapsed(s)	CPU(s)	Gap%	Valid (BKS=450)
Multi stage	548.56	548.56	0.00	0.00	0.00	21.9%	True
Tabu search	464.48	477.73	9.41	0.04	0.04	3.2%	True
SA	463.39	475.30	9.11	0.03	0.03	3.0%	True
ALNS	456.34	456.88	0.76	0.02	0.02	1.4%	True
ACO	451.95	454.32	2.05	0.08	0.08	0.4%	True
GA island	451.95	451.95	0.00	0.70	0.69	0.4%	True
B&B(LDS)	499.77	499.77	0.00	0.00	0.00	11.1%	True
problem: E-n22-k4.vrp (customers=21, vehicles=4, capacity=6000)							
Algorithm	Best	Mean	Std	Elapsed(s)	CPU(s)	Gap%	Valid (BKS=375)
Multi stage	512.51	512.51	0.00	0.00	0.00	36.7%	True
Tabu search	508.22	509.27	0.78	0.04	0.04	35.5%	True
SA	483.60	487.97	5.70	0.02	0.03	29.0%	True
ALNS	383.87	411.80	27.10	0.03	0.02	2.4%	True
ACO	406.24	414.27	10.95	0.15	0.15	8.3%	True
GA island	383.65	389.64	4.58	0.96	0.96	2.3%	True
B&B(LDS)	510.51	510.51	0.00	0.00	0.00	36.1%	True
problem: A-n32-k5.vrp (customers=31, vehicles=5, capacity=100)							
Algorithm	Best	Mean	Std	Elapsed(s)	CPU(s)	Gap%	Valid (BKS=784)
Multi stage	1497.50	1497.50	0.00	0.00	0.00	91.0%	True
Tabu search	981.69	997.55	14.40	0.08	0.09	25.2%	True
SA	971.98	1024.47	40.46	0.06	0.05	24.0%	True
ALNS	833.04	875.31	30.28	0.05	0.05	6.3%	True
ACO	920.61	949.84	20.84	0.31	0.32	17.4%	True
GA island	885.62	921.92	30.25	1.33	1.31	13.0%	True
B&B(LDS)	1025.03	1025.03	0.00	0.01	0.00	30.7%	True
problem: A-n80-k10.vrp (customers=79, vehicles=10, capacity=100)							
Algorithm	Best	Mean	Std	Elapsed(s)	CPU(s)	Gap%	Valid (BKS=1763)
Multi stage	3540.48	3540.48	0.00	0.00	0.00	100.8%	True
Tabu search	2897.27	2958.28	43.39	0.11	0.10	64.3%	True
SA	2658.35	2874.80	158.42	0.07	0.07	50.8%	True
ALNS	1927.81	1956.09	20.12	0.20	0.20	9.3%	True
ACO	2199.60	2239.80	28.55	1.45	1.44	24.8%	True
GA island	2414.69	2436.06	19.51	3.07	3.06	37.0%	True
problem: X-n101-k25.vrp (customers=100, vehicles=25, capacity=206)							
Algorithm	Best	Mean	Std	Elapsed(s)	CPU(s)	Gap%	Valid (BKS=27591)
Multi stage	43374.16	43374.16	0.00	0.00	0.00	57.2%	False
Tabu search	46924.07	52148.12	3714.09	0.17	0.17	70.1%	False
SA	46363.43	47389.33	765.62	0.12	0.12	68.0%	False
ALNS	40373.03	41661.15	1086.65	0.38	0.39	46.3%	False
ACO	36725.05	37224.89	407.37	1.92	1.92	33.1%	False
GA island	40627.14	41493.95	662.08	3.02	3.02	47.2%	False
problem: M-n200-k17.vrp (customers=199, vehicles=17, capacity=200)							
Algorithm	Best	Mean	Std	Elapsed(s)	CPU(s)	Gap%	Valid (BKS=1275)
Multi stage	5572.46	5572.46	0.00	0.00	0.01	337.1%	True
Tabu search	3885.12	4080.71	141.92	0.26	0.26	204.7%	True
SA	4070.52	4127.12	77.54	0.20	0.20	219.3%	True
ALNS	1488.73	1555.28	63.58	1.11	1.10	16.8%	True
ACO	1874.45	1910.95	25.92	9.25	9.23	47.0%	True
GA island	2575.72	2635.19	42.84	9.47	9.44	102.0%	True

א. ניתן לראות הבדלים לפי גודל המופע:

- ALNS שולט במופעים הבינוניים והגדולים: הוא מנצח ב A-n80, A-n32, M-n200 שהוא המופע הגדול ביותר. מחזור ההרס והתיקון מאפשר לאלגוריתם לשבור טופולוגיות שגויות של ניתוב ולקבץ לקוחות מחדש בצורה יעילה.
- GA island מנצח במופעים הקטנים: מנצח ב E-n222 ושוויון ב P-n16. השילוב של קרוסאובר ממטי וחיפוש מקומי ממוקד מצוין לבעיות בעלות עץ חיפוש רדוד ביחס לאחרים.
- ACO מנצח את המופע הקשה ביותר, ב X-n101-k25 הוא הגיע לעלות הטובה ביותר, וכן על המופע הזה ארחיב בהמשך. וכן הוא משווה את הפתרון של GA על P-n16.
- ההיוריסטיקה הרב שלבית מהווה פתרון בסיס טוב, היא המהירה ביותר אבל לה העלויות הגבוהות ביותר, כצפוי.

ב. זמני ריצה והטריידאופים:

העמודה של זמני הריצה מראה את העיקרון של עלות מול חקר. היוריסטיקת הבסיס מסיימת כמעט באפס זמן. אלגוריתמי החיפוש המקומי ALNS, Tabu, SA, נותרו מהירים מאוד גם על 199 לקוחות וזאת בשל הגישה המהירה למטריצת המרחקים. ובניגוד אליהם GA, ACO, היו היקרים ביותר חישובית, כי כאן מדובר בנמלים שבונות מסלול מאפס, וחישוב ממטי על פני איים רבים, אבל מספקים עמידות גבוהה כנגד מינימום מקומיים.

ג. אילוץ הקיבולת והחריגה ב X-n101-k25:

המופע החריג דורש צי של 25 רכבים, אשר מהווה לנו חסם עליון לביקוש. בכל ריצות האלגוריתמים, החיפוש הסתיים עם 26 מסלולים במקום 25, כמובן שהיה מענה מלא לכל הלקוחות וללא חריגות קיבולת ברכבים, ולכן נכשל פורמלית וסומן ב Valid- false.

תופעה זו כפי שלמדנו היא אינה שגיאה או באג, אלא תוצר של טופולוגיית עץ החיפוש שנגזרת מאופרטור Relocate. מופע זה הוא בעיית אריזה, כיוון שהיוריסטיקת האתחול פזרה את הלקוחות ויצרה ניסיון של הזזת הלקוח ממשאית 26 בחזרה למשאיות 1-25 עם העברה בודדת, אשר הניב הרבה קנסות, כך שהאלגוריתם דחה. אציין כי בחמשת המופעים האחרים, כולל המופע הגדול פי 2 מזה הושגה חוקיות מלאה.

ד. Gap:

- GA ו ACO במופעים הקטנים מציגים קרבה טובה ל BKS, בפער של 0.4% ב P-n16 ופער של 2.3% ב E-n222 ובכך מראה על היעילות שלהם במופעים הקטנים.
- Multi stage: עמודת Gap מראה איך איכות הפתרון החמדני מתדרדרת ככל שעולה גודל הבעיה, מפער של 21.9% ב P-n16 עד ל 337% במופע הגדול M-n200.
- ALNS בגדולים שומר על פער נמוך ויציב ביחס ל BKS-6.3% ב A-n32, 9.3% ב A-n80 ו 16% ב M-n200, ובכך מראה על הדומיננטיות שלו למופעים גדולים שכאלה.

8. ויזואליזציה:

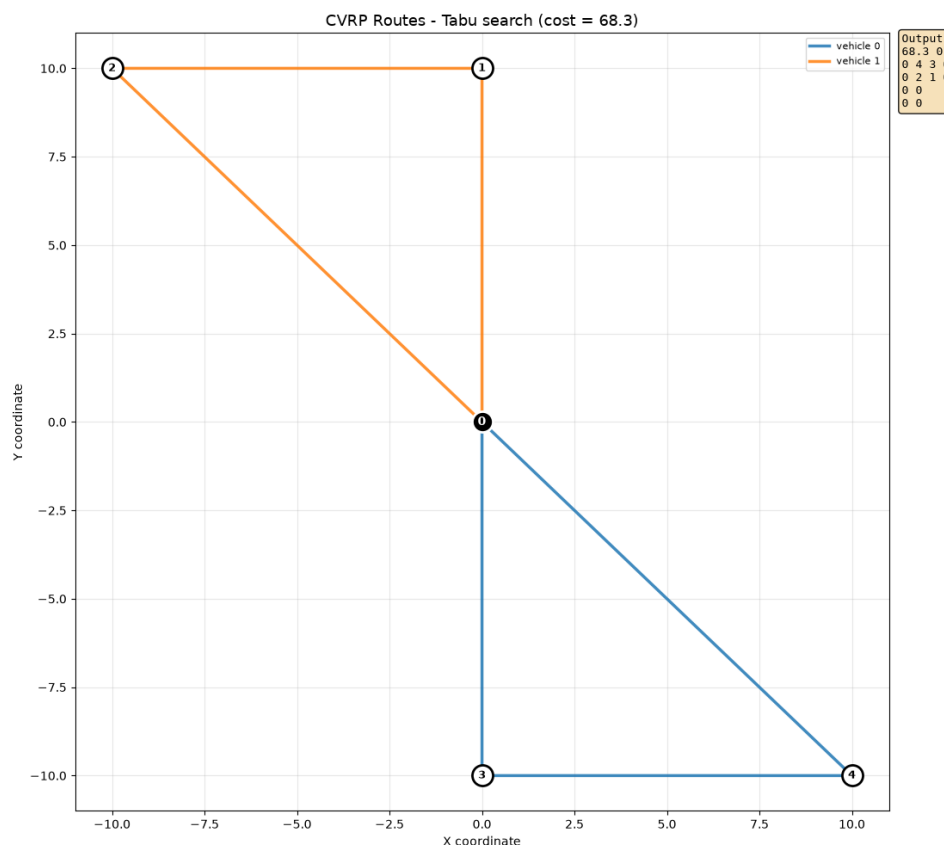
עבור כל מופע יצרתי 2 המחשות:

- מפת ניתוב: מציגה את המחסן, צמתי הלקוחות ונתיבי הרכבים שכל אחד בצבע אחר. בלוח נפרד בצד הימני הצגתי את מטריצת המסלולים.
- גרף השוואת ביצועים: מציג את העלות הממוצעת של כל אלגוריתם ממיון מהטוב לרע, כולל פסי שגיאה שמייצגים את התוצאות בין ה seeds השונים, ונקודה אדומה שמייצגת את הפתרון הטוב ביותר שנמצא.

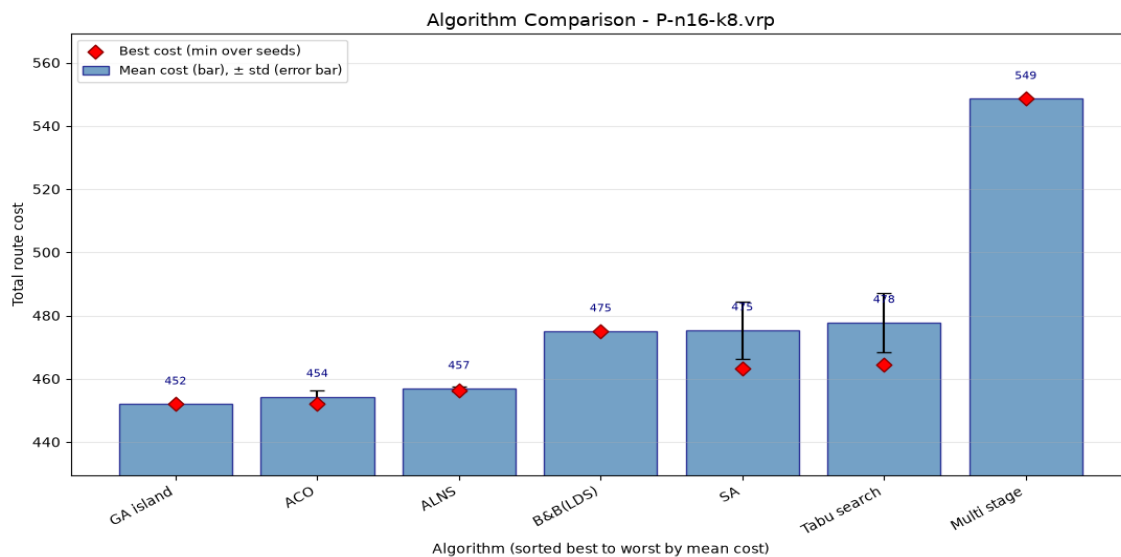
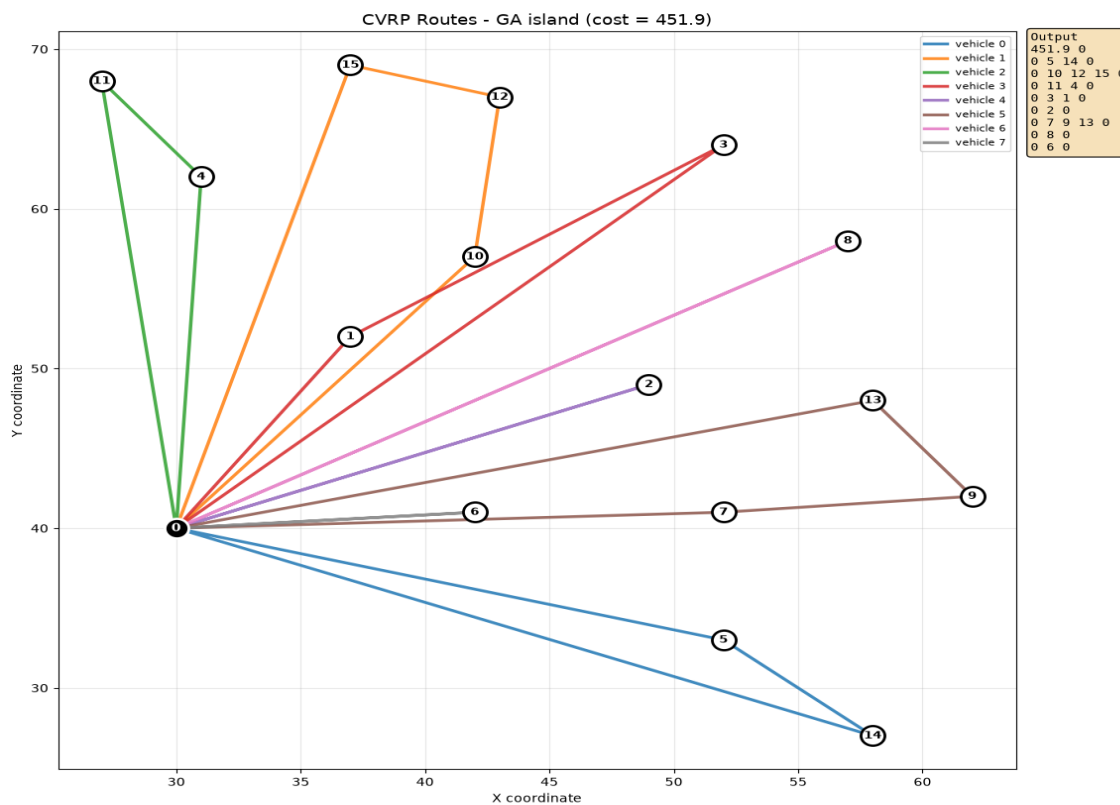
ראשית נתחיל מהדוגמה בתרגיל: (כולל הטבלה של התוצאות)
 אציין כי הפתרון שהוצג במטלה, בעלות 80.645. מטא-יוריסטיקות החיפוש המקומי שמימשתי ALNS, SA, TABU, חלצו את האופטימום האמיתי של מופע זה בעלות 68.28 המושגת על ידי חלוקת הלקוחות לשני מסלולים משולשים סימטריים ונפרדים, ללא הקפיצה האלכסונית בלוח שקיימת באיור המטלה.

problem: example-n5-k4 (customers=4, vehicles=4, capacity=10)

Algorithm	Best	Mean	Std	Elapsed(s)	CPU(s)	Gap%	Valid (BKS=80.645)
Multi stage	82.43	82.43	0.00	0.00	0.00	2.2%	True
Tabu search	68.28	68.28	0.00	0.02	0.02	-15.3%	True
SA	68.28	68.28	0.00	0.01	0.02	-15.3%	True
ALNS	68.28	68.28	0.00	0.01	0.01	-15.3%	True
ACO	80.64	80.64	0.00	0.01	0.01	-0.0%	True
GA island	80.64	80.64	0.00	0.34	0.34	-0.0%	True
B&B(LDS)	80.64	80.64	0.00	0.00	0.00	-0.0%	True

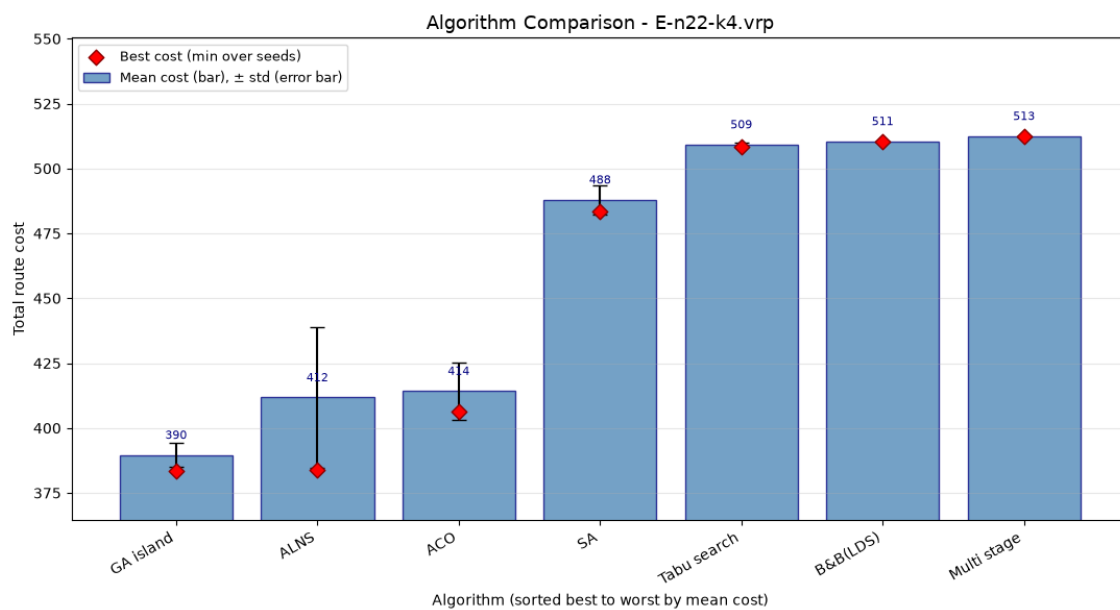
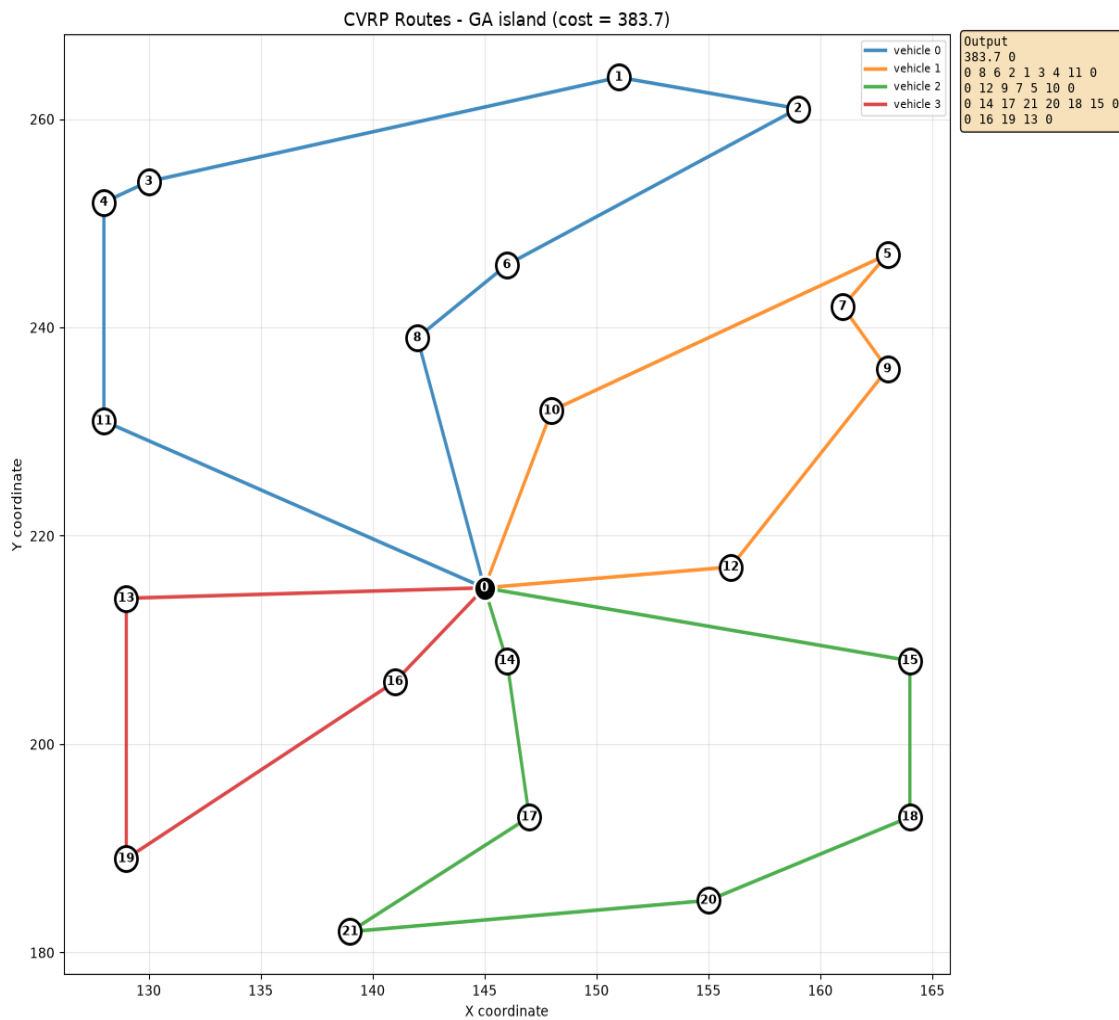


- **P-n16-k8**, הטוב ביותר: GA & ACO שהשיגו אותו דבר, עלות 451.95, 8 מסלולים:



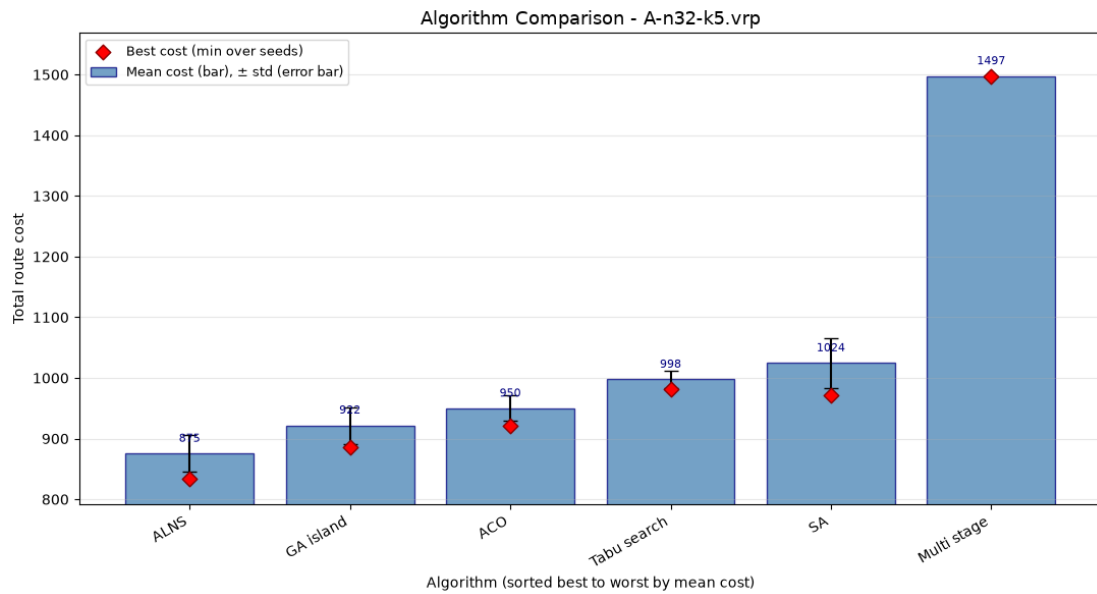
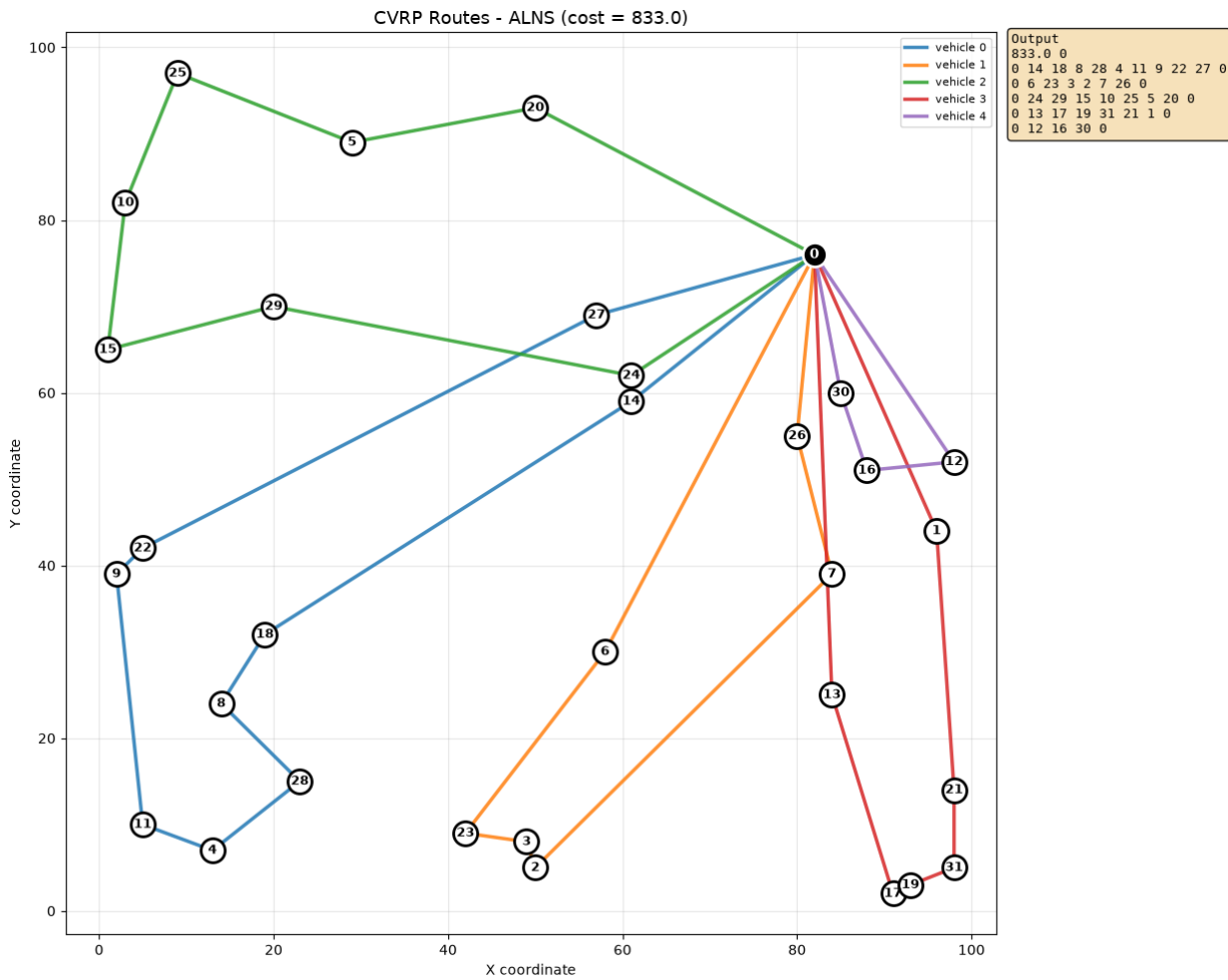
8 המסלולים נפרסים בצורה טובה מתוך המחסן המרכזי, במופע הכולל 15 לקוחות בלבד, GA מייצר מסלולים קצרים וללא הצטלבויות, המגיעים לאיכות הטובה ביותר.

- E-n22-k4, הטוב ביותר: GA island, עלות 383.65, 4 מסלולים:



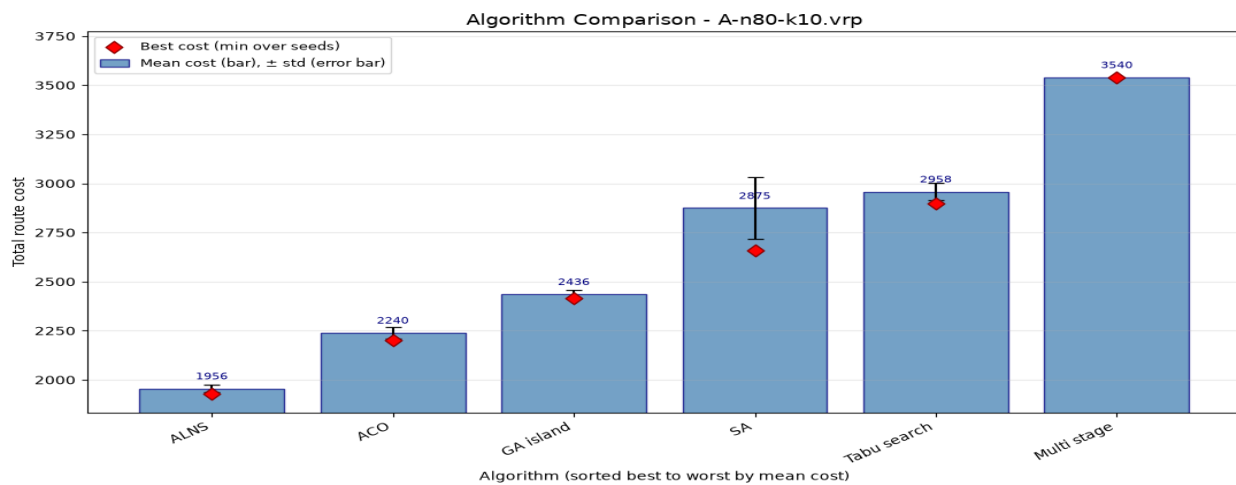
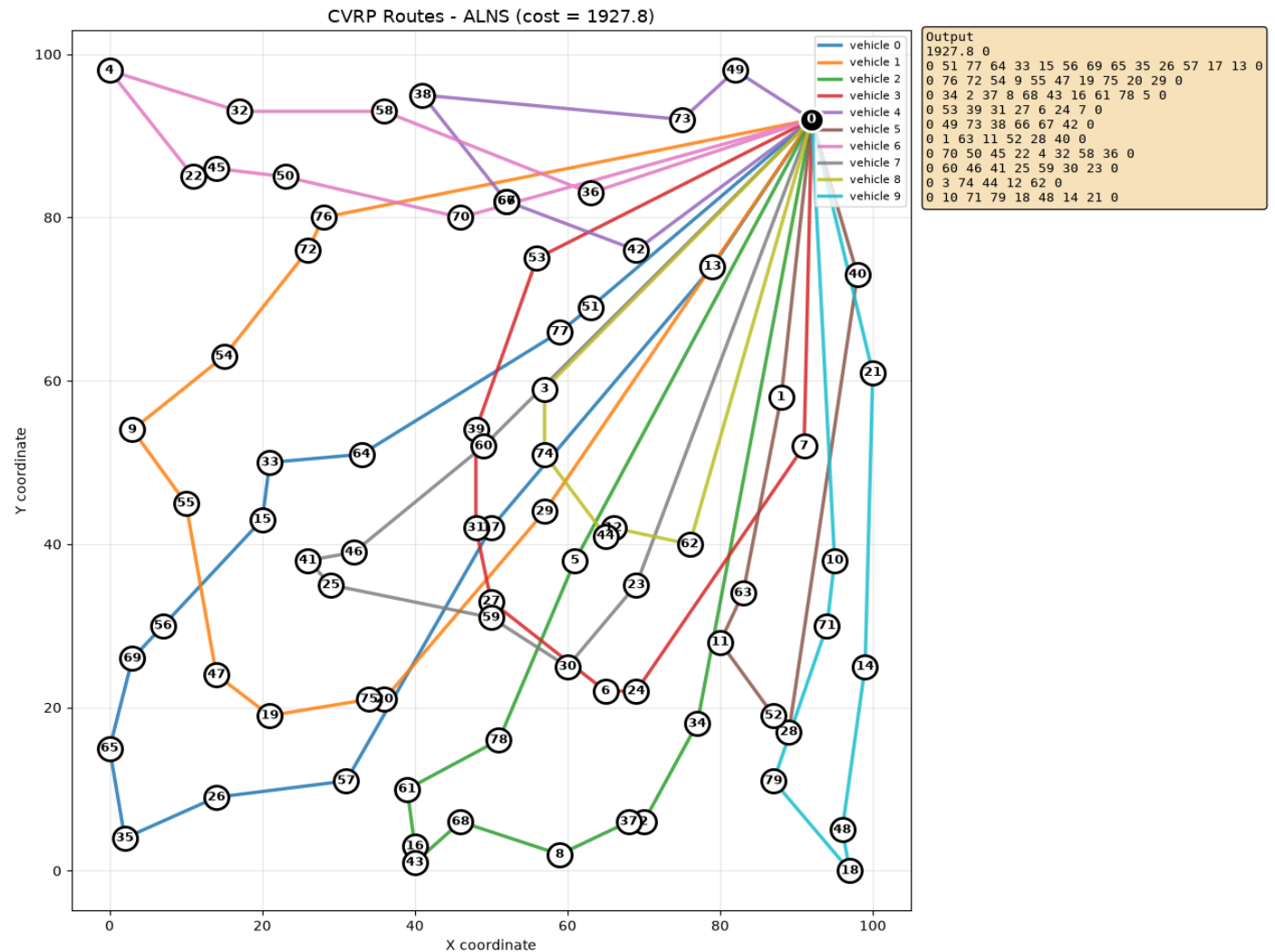
4 מסלולים משרתים את כל 21 הלקוחות תחת מגבלת הקיבולת הגדולה, 6000. ניתן לראות כיצד GA מקבץ לקוחות קרובים גיאוגרפית למסלולים ארוכים ומסודרים, ללא חפיפה או חיתוכים ביניהם.

- **A-n32-k5**, הטוב ביותר: ALNS, עלות 5,833.04 מסלולים:



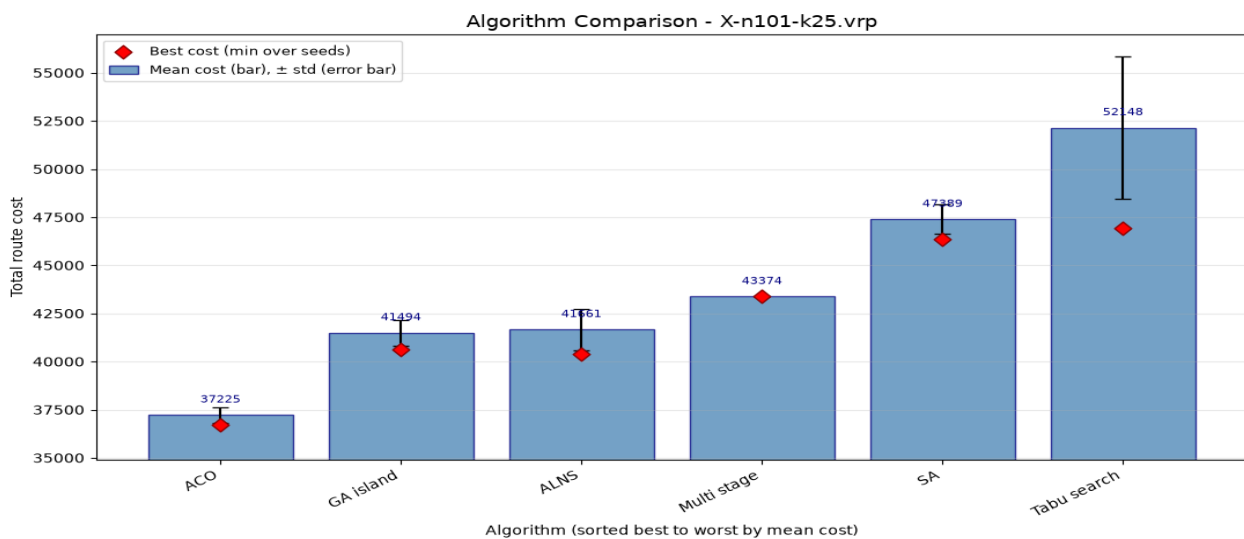
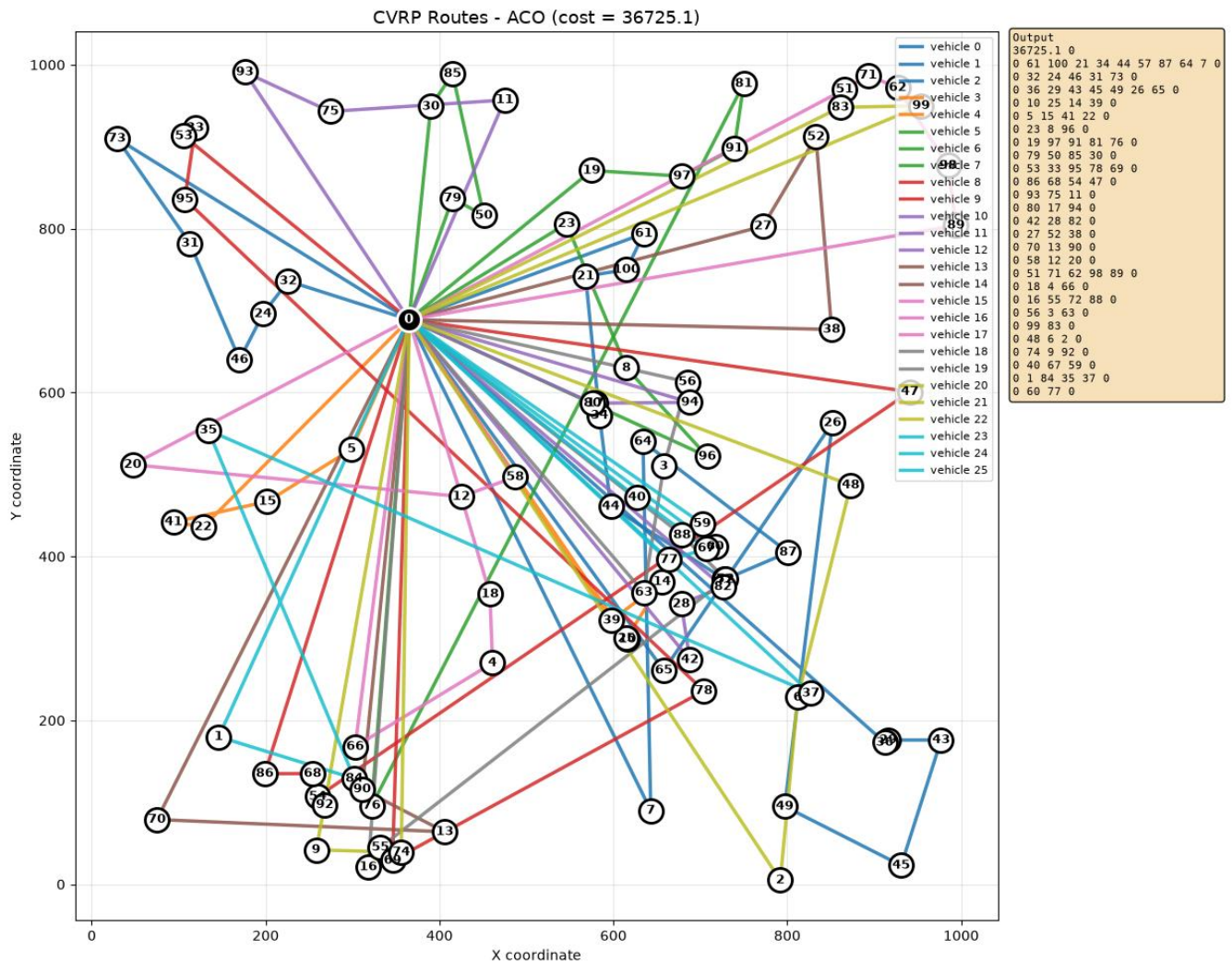
ALNS מחלק את 31 הלקוחות ל 5 מסלולים מאוזנים ומופרדים בצורה טובה מבחינת מיקום.

- **A-n80-k10**, הטוב ביותר: ALNS, עלות 1927.81, 10 מסלולים:



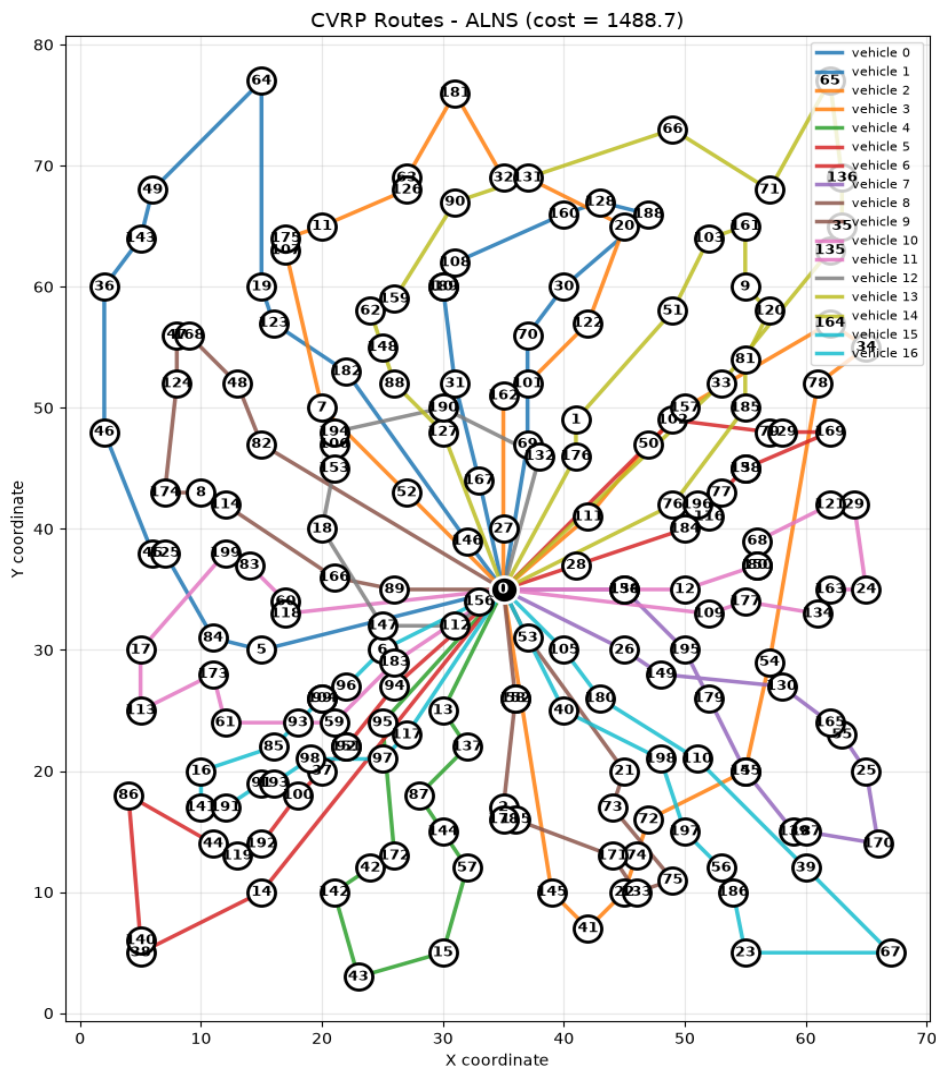
למרות שהוגדל המופע ל 79 לקוחות, ALNS מצליח לשמור על 10 המסלולים כשהם מופרים בצורה יחסית נקייה ומובנת, כך שממחיש את היעילות של ההרס והתיקון בארגון מופעים מורכבים.

- X-n101-k25, הטוב ביותר: ACO, עלות 36725.05, 26 מסלולים:

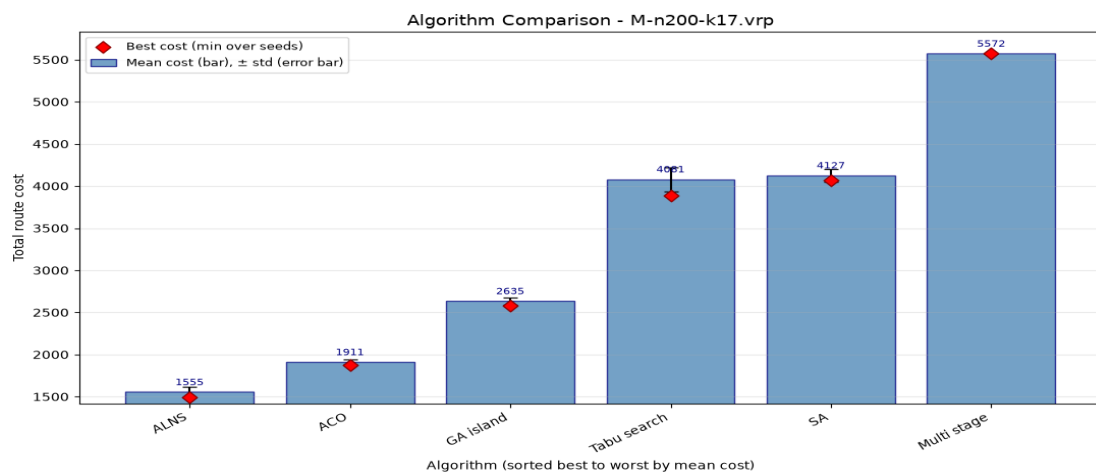


במופע זה, 26 המסלולים, ומסלול אחד מעבר לחסם של 25, נראים דחוסים וארוזים בצפיפות רבה סביב המחסן. המפה מראה שאין מרווח גיאוגרפי פנוי שיוכל לספוג את המסלול העודף, מה שממחיש ויזואלית את הקושי שנידון מקודם.

- M-n200-k17, הטוב ביותר: ALNS, עלות 1488.73, 17 מסלולים:



Output
1488.7 0
0 69 70 30 188 128 160 108 189 10 31 167 0
0 5 84 125 45 46 36 143 49 64 19 123 182 146 0
0 27 162 101 122 20 131 32 181 63 126 11 175 107 7 52 0
0 111 50 157 164 34 78 54 4 72 74 22 41 145 0
0 13 137 87 144 57 15 43 142 42 172 95 0
0 28 184 196 116 77 3 158 169 129 79 102 0
0 14 38 140 86 44 119 192 100 37 92 151 94 0
0 138 195 179 155 139 187 170 25 55 165 130 149 26 0
0 82 48 168 47 124 174 8 114 166 89 0
0 53 21 73 75 133 171 115 178 2 152 58 0
0 183 59 93 61 173 113 17 199 83 60 118 0
0 154 12 80 150 68 121 29 24 163 134 177 109 0
0 112 147 18 153 106 194 190 132 0
0 76 185 81 120 9 161 103 51 1 176 0
0 33 135 35 136 65 71 66 90 159 62 148 88 127 0
0 156 6 96 104 99 85 16 141 191 91 193 98 97 117 0
0 105 180 110 39 67 23 186 56 197 198 40 0



למרות שמדובר במופע הגדול ביותר של 199 לקוחות, ALNS מצליח לשמור על 17 המסלולים מופרדים גם כן בצורה נקייה יחסית לגודל הבעיה, זה מראה את יכולת ההתרחבות של השיטה, שמצליחה לארגן עשרות לקוחות מבלי שהמסלולים יסודרו בצורה מבולגנת.

9. מסקנות:

- א. ALNS הוכח כחזק והיציב ביותר, בזאת שמנצח את כל המופעים של הבעיות הגדולות והבינוניות, ושומר על אחוזי GAP נמוכים ויציבים ביחס ל BKS, בזכות כך שיכול לבצע שינויי מבנה משמעותיים בעזרת ההרס והתיקון.
- ב. GA ממטי היה טוב מאוד בבעיות הקטנות להן עצי חיפוש נמוכים, זאת בקרבה של פחות מאחוז ל BKS, ועוד ACO טוב מאוד במציאת פתרון בסביבות קשות, כך שההתאמה של האלגוריתם תלויה בגודל ובאילוצי הבעיה.
- ג. Strategic oscillation היה הכרחי כאן לבעיות הניתוב, בכך שנתנו היתר לשימוש במרחב הפתרון הלא חוקי עם פונקציות קנס רכות משחרר את האלגוריתם מחסימות קשות.
- ד. טריידאוף בין עלות חישוב לבין איכות הפתרון, מציאת פתרונות מורכבים בGA וACO דרשה משאבים משמעותית יותר מהפתרונות החמדניים.
- ה. המופע X-n101 הראה כי אלגוריתמים שמבוססים על אופרטור relocate בסיסי נכשלים בבעיות הדוקות כאלו, ונדרש תכנון סביבות שכונה אחר מהנוכחי עבור מופעים כאלו קיצוניים.
- ו. בדיקת שפיות-Ackley, הוכיחה שצריך מטא יוריסטיקות, בעוד שכל 5 המטא יוריסטיקות התכנסו למינימום הגלובלי במרחב רציף, האלגוריתם החמדן נלכד במינימום מקומי, וזה מראה כי שיטה חמדנית ללא מנגנונים של חקירה וחילוץ מוגבלת בכל מרחב.

• חלק ב: Rush hour GP/GEP

1. מבוא:

בחלק זה השתמשתי בתכנות גנטי, GP/GEP, זאת כדי לפתח באופן אוטומטי פונקציות יוריסטיות עבור האלגוריתם A^* שבניתי בתרגיל הראשון לפתרון בעיות RH. פאזל זה הוא משחק בלוקים מחליקים על לוח בגודל 6×6 , כך שמגוון רכבים ומשאיות חוסמים את הלוח, והמטרה להחליק רכב אדום לעבר היציאה. אלגוריתם ה- A^* סורק את מרחב תצורות הלוח, ואיכות הפונקציה היוריסטית היא שתקבע כמה מצבים יאלץ לפתח לפני מציאת הפתרון.

בחלק זה במקום תכנון ידני של ההיוריסטיקה, אפשרתי לאבולוציה לגלות אותה. במקום שיקול דעת שלי שבוחן איזה מאפיינים הם החשובים בלוח, ה- GP ו- GEP סורקים את מרחב הנוסחאות היוריסטיות האפשריות ובוררים את אלו שגורמות ל- A^* לפתור במינימום מאמץ חיפוש.

כך ש- GP/GEP מייצרים יוריסטיקה מועמדת, A^* מורץ איתה על מספר פאזלים, ביצעו נמדדים, ומדידה זו מהווה את ציון הפיטנס של הפרט באוכלוסייה הגנטית, כך שמה שגורם ל- A^* לפתח פחות מצבים נחשב לכשיר יותר ועם סיכוי גבוה יותר להתרבות.

2. התשתית מתרגיל 1:

ה- GP ו- GEP כאן מבוססים על פתרון תרגיל 1:

- State: מחלקה שמייצגת תצורה בודדת של הלוח, תומכת ביצירת כל המהלכים הבאים, ונותנת מפתח גיבוב שמאפשר ל- A^* לזהות מצבים שביקר בהם.
- חיפוש A^* : מנהל תור עדיפויות הממוין לפי $f = g + h$ כך ש- g הוא מספר המהלכים שבוצעו עד כה, ו- h מהווה הערכה יוריסטית למספר המהלכים שנותרו. בכל פעם יפתח את המצב עם f הנמוך ביותר. ובמקרה של התרגיל הנוכחי הפונקציה לא מחזירה רק את מסלול הפתרון, אלא גם את כמות המצבים שפותחו, N , אורך המסלול, d , זמן הריצה ודגל הצלחה. כמות המצבים שפותחו זה מה ש- GP מנסה למזער.
- היוריסטיקות מתרגיל 1: כאן מהוות את הבסיס לעצי GP ולמחרוזות GEP :
 - $H1$ אשר סופרת את הרכבים החוסמים ישירות את נתיב הרכב האדום ליציאה.
 - $H2$ הסופרת גם את הרכבים שחוסמים את החוסמים.
 - בנוסף צרפתי את $t_dist_to_exit$ שמודדת את המרחק מהיציאה.

3. רכיבי GP:

א. ייצוג העץ (abstract syntax tree - AST):

בתכנות גנטי, פרטים באוכלוסייה מיוצגים כעצי תחביר מופשטים- עצי ביטויים המורכבים מצמתים פנימיים (functions) ומעלים (terminals). עץ מחושב לערך מספרי על ידי הערכה רקורסיבית של תתי העצים שלו והפעלת הפונקציה המתאימה בכל צומת פנימי.

בקוד כל פרט בGP מיוצג כעץ של אובייקטים מסוג GPNode. קבוצת הפונקציות, צמתים פנימיים, היא ADD, SUB, MUL, MAX, אלו אופרטורים אריתמטיים שמשלבים תתי ביטויים. וקבוצת העלים כוללת את H1, H2, DistExit, 1.0, 2.0. כאשר עץ מוערך על מצב לוח מסוים, כל עלה מחזיר את ערכו עבור אותו מצב, למשל העלה H2 מחזיר את ספירת חוסמי החוסמים, וכל פונקציה פנימית משלבת את ערכי ילדיה. כך שעץ כמו MUL(2.0, H2) מעריך H2 המקורי עבור כל מצב. כך שהעיצוב הזה מאפשר התפתחות אבולוציונית, ע"י שילוב כל אלו, ה GP יכול לבטא מרחב גדול של נוסחאות היוריסטיות מהפשטות כמו H1 עד למורכבות יותר.

```
class GPNode:
    """
    Node in the abstract syntax tree- AST
    represents operator (ADD, MAX...) if is an internal node,
    or a feature/constant (H1, 2.0...) if is a terminal leaf
    """
    def __init__(self, is_terminal, value, left=None, right=None):
        self.is_terminal = is_terminal
        self.value = value
        self.left = left
        self.right = right

    def evaluate(self, state):
        """
        recursively evaluates the AST to compute the heuristic value for a state
        """
        if self.is_terminal:
            if callable(self.value):
                return self.value(state)
            return self.value

        left_val = self.left.evaluate(state)
        right_val = self.right.evaluate(state)

        if self.value == 'ADD':
            return left_val + right_val
        elif self.value == 'SUB':
            # Bounded to 0 to prevent negative heuristics, keeping A* well-behaved.
            return max(0.0, left_val - right_val)
        elif self.value == 'MUL':
            return left_val * right_val
        elif self.value == 'MAX':
            return max(left_val, right_val)
        return 0
```

ב. אתחול בשיטת Grow:

צריך לייצר עצי GP עבור האוכלוסייה הראשונית, ושתי השיטות לכך הן: Full בה כל ענף גד עד לעומק המרבי, ומייצר עץ מלא, סבוך וסימטרי, והשנייה היא Grow, לפיה ענפים יכולים להסתיים מוקדם על ידי בחירת עלה, דבר המייצר עצים לא סימטריים בעומקים שונים. למדנו כי Grow מייצרת גיוון מבני גדול יותר בדור הראשון.

בקוד פונקציית יצירת העצים בונה אותם בשיטת Grow, כל שבכל צומת קיימת הסתברות אקראית לבחור פונקציה ולהמשיך ברקורסיה לבניית תתי העצים, או לבחור עלה ולסיים את הענף, זאת בכפוף לעומק מרבי. כך מייצר אוכלוסייה ראשונית עם מבנים וגדלים מגוונים.

```
def generate_random_tree(max_depth, current_depth=0):
    if current_depth >= max_depth or (current_depth > 0 and random.random() < 0.3):
        return GPNode(is_terminal=True, value=random.choice(terminals))

    op = random.choice(functions)
    left = generate_random_tree(max_depth, current_depth + 1)
    right = generate_random_tree(max_depth, current_depth + 1)
    return GPNode(is_terminal=False, value=op, left=left, right=right)
```

ג. אופרטורים גנטיים: ב GA מפתחים את האוכלוסייה עם אופרטורים מתאימים למבני עצים:

- Subtree crossover: לוקח שני עצי הורים, בוחר תת עץ אקראי בכל אחד מהם, ומחליף ביניהם. פעולה זו מייצרת צאצאים שיורשים תתי מבנים מוצלחים משני ההורים.

```
def crossover_trees(parent1, parent2):
    child = copy.deepcopy(parent1)
    nodes_in_child = []

    def traverse_child(n):
        nodes_in_child.append(n)
        if not n.is_terminal:
            traverse_child(n.left)
            traverse_child(n.right)

    traverse_child(child)
    target_node = random.choice(nodes_in_child)
    donor_node = copy.deepcopy(get_random_node(parent2))

    # swap the contents of the target node with the donor
    target_node.is_terminal = donor_node.is_terminal
    target_node.value = donor_node.value
    target_node.left = donor_node.left
    target_node.right = donor_node.right

    return child
```

- Subtree mutation: בוחר תת עץ אקראי ומחליף אותו בתת עץ חדש שנוצר באופן אקראי, ובכך מזריק מטעם גנטי חדש לאוכלוסייה.

```
def mutate_tree(node, max_depth, current_depth=0):  
    if random.random() < 0.15 or current_depth >= max_depth:  
        return generate_random_tree(max_depth - current_depth)  
  
    new_node = copy.deepcopy(node)  
    if not new_node.is_terminal:  
        if random.random() < 0.5:  
            new_node.left = mutate_tree(new_node.left, max_depth, current_depth + 1)  
        else:  
            new_node.right = mutate_tree(new_node.right, max_depth, current_depth + 1)  
    return new_node
```

- וכן במימוש עצמו יש שמירה על בחירת הטורניר ועל אליטיזם, כך שהנוסחה הטובה ביותר שנמצאה לא תאבד.

ד. פונקציית Fitness:

ידוע לנו כי קיימת בעיית ההתנפחות בתכנות גנטי ובה העצים גדלים ללא שליטה לאורך הדורות ומתמלאים בקוד חסר משמעות, שלא משפיע על הקלט אבל פוגע בזמני הריצה ובקריאות הנוסחה.

בקוד, כדי לפתח יוריסטיקה שהיא אפקטיבית ויעילה, הגדרתי פונקציית פיטנס המענישה לפי שלושה דברים:

- עונש כבד על כשלון בפתרון הפאזל- קנס של 5000, שכבר גורם לסינון מיידי של יוריסטיקות לא מתאימות, שגורמות ל-A* לקרוס או לחרוג מזמן הריצה.
- קנס על ממוצע המצבים שפותחו, N, דבר הדוחף את GP למזער ישר את מאמץ החיפוש.
- מופעל קנס סיבוכיות פרופורציונלי לגודל העץ, size*5, קנס זה נלחם בהתנפחות בכך שהוא הופך עצים גדולים ומסורבלים לבעלי קשירות נמוכה בהשוואה לעצים קומפקטיים שמשיגים אותה תוצאה. וזה מוביל את האבולוציה לבחור נוסחאות קצרות ומהירות, ובכך נותן תגמול ליוריסטיקות בעלות סיבוכיות חישובית נמוכה.

וכן תחמתי את SUB שיחזר את המקסימלי בין הערך שמצא ל 0 כדי שלא יחזיר ערכים שליליים, שכן h שלילי ישבש את תור העדיפויות של A* ויפגע בתקינות האלגוריתם.

```
def evaluate_gp_fitness(tree, training_boards):
    total_nodes_expanded = 0

    def h_func(state):
        return tree.evaluate(state)

    for board in training_boards:
        #time limit to 1 second, to kill bloated/infinite heuristics
        res = a_star_search(board, h_func, time_limit=1.0)
        # using the keys from original a_star_search
        if res.get("Success (Y/N)") == "Y":
            total_nodes_expanded += res["N"]
        else:
            # big penalty for failing to solve
            total_nodes_expanded += 5000

    avg_nodes = total_nodes_expanded / len(training_boards)

    # bonus for low computational complexity, so deep or complex trees receive a penalty
    complexity_penalty = tree.get_size() * 5.0

    return avg_nodes + complexity_penalty
```

4. רכיבי GEP:

- וכן ממשתי גם אלגוריתם GEP. והשוני המהותי מ GP הוא באופן ייצוג הפרט, שבמקום עץ דינאמי שיכול להתנפח, פרט ב GEP מיוצג כמחרוזת ליניארית באורך קבוע, גנום, שמחולקת לשני חלקים:
- ראש שיכול להכיל פונקציות או טרמינלים.
 - זנב שמכיל רק טרמינלים.

אורך הזנב נקבע לפי הכלל: $t = h * (n - 1) + 1$ כאשר n הוא מספר הפרמטרים לפונקציה (האריות). וכיוון שכל הפונקציות שלי הן בינאריות, $n = 2$, הזנב תמיד מאורך $h + 1$, חוקיות זו מיושמת בפונקציה `gep_create_genome`, ומבטיחה שכל גן, לא משנה התוכן האקראי של הראש, יתורגם תמיד לעץ ביטוי שלם וחוקי.

```
def gep_create_genome(head_length):  
    tail_length = head_length * (GEP_MAX_ARITY - 1) + 1  
    head = [gep_random_head_symbol() for _ in range(head_length)]  
    tail = [random.choice(terminals) for _ in range(tail_length)]  
    return head + tail
```

- א. תרגום הגנום לעץ- `gep_translate`: פונקציה הקוראת את הגנום הלינארי משמאל לימין ומתרגמת אותו לעץ מסוג GPNode עם BFS, ומנהלת תור של צמתים הממתינים לילדים. בניה זו מאפשרת ל GEP לעבוד עם אותו אופן חישוב, `evaluate_gp_fitness`, של GP דרך הפונקציה המתאימה אותה `evaluate_gep_fitness`, דבר שמבטיח השוואה אובייקטיבית עם הבסיס של אותה פונקציית הערכה.

```
def gep_translate(genome):  
    # create the root node from the first symbol  
    def make_node(symbol):  
        #function symbol  
        if isinstance(symbol, str):  
            return GPNode(is_terminal=False, value=symbol)  
        return GPNode(is_terminal=True, value=symbol)  
  
    root = make_node(genome[0])  
    #queue of nodes still waiting for their children  
    queue = [root]  
    idx = 1  
    while queue:  
        node = queue.pop(0)  
        if not node.is_terminal:  
            # binary function needs exactly two children  
            for _ in range(GEP_MAX_ARITY):  
                if idx >= len(genome):  
                    # for safety- fill with a terminal  
                    child = GPNode(is_terminal=True, value=random.choice(terminals))  
                else:  
                    child = make_node(genome[idx])  
                    idx += 1  
                if node.left is None:  
                    node.left = child  
                else:  
                    node.right = child  
                queue.append(child)  
    return root
```

ב. אופרטור מוטציה- gep_mutate: מוטציה נקודתית הפועלת על המחרוזת הלינארית. וכדי לשמר את חוקיות הגנום, המימוש בודק את אינדקס המיקום: אם המוטציה מתרחשת בראש, התו יכול להשתנות לכל פונקציה או טרמינל, אבל אם בזנב התו ישתנה לטרמינל אחר בלבד.

```
def gep_mutate(genome, head_length):  
    """  
    point mutation- each position may change. A head position can mutate into any symbol (function or terminal),  
    a tail position may mutate into a terminal only, so the gene stays valid  
    """  
    new_genome = genome[:]  
    for i in range(len(new_genome)):  
        if random.random() < 0.15:  
            if i < head_length:  
                new_genome[i] = gep_random_head_symbol()  
            else:  
                new_genome[i] = random.choice(terminals)  
    return new_genome
```

ג. אופרטור הצלבה- gep_crossover: crossover לנקודה אחת המבוצע כחיתוך והחלפת מקטעים על המחרוזת הלינארית. בזכות האורך הקבוע והמבנה הזהה של שני ההורים, הצאצאים שנוצרים הם תמיד גנומים חוקיים. וכן ההצלבה והמוטציה זולים חישובית לעומת החלפת תתי עצים ב GP.

```
def gep_crossover(parent1, parent2):  
    """  
    one point crossover on the linear genome: pick a cut point and swap the tails of the two parents.  
    Because both parents share the same length and head/tail structure, the children are valid genomes.  
    """  
    if len(parent1) < 2:  
        return parent1[:], parent2[:]  
    point = random.randint(1, len(parent1) - 1)  
    child1 = parent1[:point] + parent2[point:]  
    child2 = parent2[:point] + parent1[point:]  
    return child1, child2
```

ד. האבולוציה: run_gene_expression_programming, מריצה את לולאת הדורות של GEP תוך כך ששומרת על אותם מנגנונים של אליטיזם ובחירת טורניר ומשתמשת בתרגום לעץ רק לצורך חישוב הפיטנס והדפסת הנוסחה הטובה ביותר.

5. אימון והערכה

כדי למנוע overfitting, GP/GEP אומן על קבוצת פאזלים אחת והוערך על קבוצה אחרת לגמרי. שלב האימון השתמש בלוחות שנוצרו לפי לוחות קושי משתנות בעומקי פתרון של 6 עד 11 צעדים, בעזרת הפונקציה ליצירת לוחות שבנינו בתרגיל הקודם. ובכך היוריסטיקה המפותחת תהיה כוללת גם ללוחות קלים וקשים, במקום ללמוד פתרון לוח ספציפי.

ההערכה הסופית בוצעה על סט הבעיות המלא מקובץ rh.txt מתרגיל 1 שאליהן לא נחשף GP/GEP במהלך האימון.

וכך בוצעה השוואה בין היוריסטיקה המקורית H2 לבין יוריסטיקה שפותחה על ידי GP ו GEP על כל 40 הבעיות. עבור כל פאזל תועד כמות הצמתים שפותחו, זמן הריצה ואורך מסלול הפתרון. וכן בדקתי אם מסלול הפתרון של GP/GEP זהה באורכו לשל H2 או שהתארך.

6. תוצאות:

:GP

```
Evolving heuristic (GP)
GP population- size: 20
Gen 01/10 | Best Fitness: 100.0 | Formula: MUL(2.0, MAX(DistExit, SUB(MUL(H2, DistExit), SUB(H1, H2))))
Gen 02/10 | Best Fitness: 88.2 | Formula: MAX(MUL(DistExit, MUL(H2, MUL(1.0, H2))), H1)
Gen 03/10 | Best Fitness: 82.4 | Formula: MUL(2.0, MUL(2.0, H2))
Gen 04/10 | Best Fitness: 69.0 | Formula: MUL(MUL(2.0, H2), H2)
Gen 05/10 | Best Fitness: 69.0 | Formula: MUL(MUL(2.0, H2), H2)
Gen 06/10 | Best Fitness: 69.0 | Formula: MUL(MUL(2.0, H2), H2)
Gen 07/10 | Best Fitness: 69.0 | Formula: MUL(MUL(2.0, H2), H2)
Gen 08/10 | Best Fitness: 69.0 | Formula: MUL(MUL(2.0, H2), H2)
Gen 09/10 | Best Fitness: 69.0 | Formula: MUL(MUL(2.0, H2), H2)
Gen 10/10 | Best Fitness: 69.0 | Formula: MUL(MUL(2.0, H2), H2)

- GP completed in 47.0s
- GP best formula: MUL(MUL(2.0, H2), H2)
```

- למדנו כי הכפלת פונקציית יוריסטיקת קבילה בקבוע הגדול מ 1 הופכת אותה ללא קבילה, שכן עשויה להעריך ביתר את העלות האמיתית שנותרה. ובכך הופכת את האלגוריתם ל A^* weighted המשנה את התנהגותו לחמדנית יותר, כך שמסתמך על היוריסטיקה ומפתח פחות מצבים, אך פוגע במציאת מסלול הפתרון הקצר ביותר. כך שיש לנו טריידאוף בין ירידה במספר הצמתים שפותחו, לבין מחיר אורך מסלול ארוך יותר.
- בפועל בתוצאות ניתן לראות כי GP הגיע לנוסחה $MUL(MUL(2.0, H2), H2)$, כלומר $2^2 \cdot (H2)$, בריצות שונות ה GA מתכנס לנוסחאות מאותו סגנון: יוריסטיקת בסיס שגדלה עם גורמי הכפלה נוספים, ובמקרה זה- העלאה בריבוע והכפלה במרחק ליציאה. זהו מבנה של יוריסטיקה לא קבילה וחמדנית.
- וכן, ללא הכוונה האבולוציה בחרה בעקרון של A^* weighted והסיקה בכך שניפוח של יוריסטיקה קבילה, מצמצם מאוד את מאמץ החיפוש, ופונקציית הפיטנס תגמלה מזעור של פיתוח מצבים ולכן היא נבחרה.

:GEP

```
Evolving heuristic (GEP)
GEP population- size: 20, head length: 6
Gen 01/10 | Best Fitness: 104.4 | Formula: MUL(MAX(MAX(1.0, H2), MAX(1.0, H2)), MUL(H2, H1))
Gen 02/10 | Best Fitness: 99.0 | Formula: MUL(MAX(MAX(1.0, H2), MAX(2.0, H2)), MUL(H2, H1))
Gen 03/10 | Best Fitness: 73.0 | Formula: MUL(DistExit, ADD(1.0, H2))
Gen 04/10 | Best Fitness: 73.0 | Formula: MUL(DistExit, ADD(1.0, H2))
Gen 05/10 | Best Fitness: 73.0 | Formula: MUL(DistExit, ADD(1.0, H2))
Gen 06/10 | Best Fitness: 73.0 | Formula: MUL(DistExit, ADD(1.0, H2))
Gen 07/10 | Best Fitness: 73.0 | Formula: MUL(DistExit, ADD(1.0, H2))
Gen 08/10 | Best Fitness: 73.0 | Formula: MUL(DistExit, ADD(1.0, H2))
Gen 09/10 | Best Fitness: 71.8 | Formula: MUL(DistExit, MUL(1.0, H2))
Gen 10/10 | Best Fitness: 71.8 | Formula: MUL(DistExit, MUL(1.0, H2))

- GEP completed in 38.8s
- GEP best formula: MUL(DistExit, MUL(1.0, H2))
```

גם GEP גילה יוריסטיקה לא קבילה מאותה משפחה שמצא GP, והתכנס לנוסחה $MUL(DistExit, H2)$ שהיא הנוסחה $MUL(DistExit, MUL(1.0, H2))$.

השוואת H2, GEP, GP:

Board	Method	Expanded (N)	Time (ms)	Path (d)
Test 05	H2 (Advanced)	594	289.9	9
Test 05	GP (Evolved)	51	22.9	11
Test 05	GEP (Evolved)	126	59.2	11
Test 10	H2 (Advanced)	1433	518.0	17
Test 10	GP (Evolved)	1018	385.8	17
Test 10	GEP (Evolved)	1170	429.4	18
Test 15	H2 (Advanced)	519	143.6	24
Test 15	GP (Evolved)	355	101.8	28
Test 15	GEP (Evolved)	370	103.0	28
Test 20	H2 (Advanced)	383	134.9	10
Test 20	GP (Evolved)	1119	385.5	20
Test 20	GEP (Evolved)	802	257.0	15
Test 25	H2 (Advanced)	6322	3195.9	28
Test 25	GP (Evolved)	3656	1694.2	38
Test 25	GEP (Evolved)	5370	2475.4	32
Test 30	H2 (Advanced)	1041	311.4	32
Test 30	GP (Evolved)	721	233.8	34
Test 30	GEP (Evolved)	938	275.8	34
Test 35	H2 (Advanced)	3895	1461.1	44
Test 35	GP (Evolved)	2689	968.0	53
Test 35	GEP (Evolved)	2920	1051.0	51
Test 40	H2 (Advanced)	2697	943.9	51
Test 40	GP (Evolved)	1646	570.8	52
Test 40	GEP (Evolved)	1657	573.0	52

Final summary on the full set (averages)

Heuristic	Avg Nodes (N)	Avg Time (ms)
H2 (Original)	1921.3	730.81
GP (Evolved)	1201.0	451.66
GEP (Evolved)	1470.3	552.58

```

formula diversity in independent runs
GP population- size: 20
Gen 01/10 | Best Fitness: 110.4 | Formula: MUL(1.0, MUL(ADD(H1, ADD(1.0, 1.0)), MUL(H2, MUL(DistExit, H2))))
Gen 02/10 | Best Fitness: 100.4 | Formula: MUL(1.0, MUL(ADD(H1, 1.0), MUL(H2, MUL(DistExit, H2))))
Gen 03/10 | Best Fitness: 100.4 | Formula: MUL(1.0, MUL(ADD(H1, 1.0), MUL(H2, MUL(DistExit, H2))))
Gen 04/10 | Best Fitness: 100.4 | Formula: MUL(1.0, MUL(ADD(H1, 1.0), MUL(H2, MUL(DistExit, H2))))
Gen 05/10 | Best Fitness: 99.0 | Formula: MUL(1.0, MUL(ADD(1.0, 1.0), MUL(H2, MUL(H2, H2))))
Gen 06/10 | Best Fitness: 98.2 | Formula: MUL(1.0, MUL(1.0, MUL(ADD(H1, DistExit), MUL(H2, H1))))
Gen 07/10 | Best Fitness: 88.2 | Formula: MUL(1.0, MUL(1.0, MUL(H2, MUL(DistExit, H2))))
Gen 08/10 | Best Fitness: 88.2 | Formula: MUL(1.0, MUL(1.0, MUL(H2, MUL(DistExit, H2))))
Gen 09/10 | Best Fitness: 88.2 | Formula: MUL(1.0, MUL(1.0, MUL(H2, MUL(DistExit, H2))))
Gen 10/10 | Best Fitness: 79.6 | Formula: MUL(H2, H2)
GEP population- size: 20, head length: 6
Gen 01/10 | Best Fitness: 89.0 | Formula: MUL(MUL(H2, MAX(MAX(H2, 2.0), 1.0)), H1)
Gen 02/10 | Best Fitness: 89.0 | Formula: MUL(MUL(H2, MAX(MAX(H2, 2.0), 1.0)), H1)
Gen 03/10 | Best Fitness: 89.0 | Formula: MUL(MUL(H2, MAX(MAX(H2, 2.0), 1.0)), H1)
Gen 04/10 | Best Fitness: 89.0 | Formula: MUL(MUL(H2, MAX(MAX(H2, 2.0), 1.0)), H1)
Gen 05/10 | Best Fitness: 84.4 | Formula: MUL(MUL(H2, MAX(H2, 1.0)), H1)
Gen 06/10 | Best Fitness: 84.4 | Formula: MUL(MUL(H2, MAX(H2, 1.0)), H1)
Gen 07/10 | Best Fitness: 80.4 | Formula: MUL(DistExit, H1)
Gen 08/10 | Best Fitness: 80.4 | Formula: MUL(DistExit, H1)
Gen 09/10 | Best Fitness: 80.2 | Formula: MUL(MUL(H2, ADD(2.0, 1.0)), H1)
Gen 10/10 | Best Fitness: 80.2 | Formula: MUL(MUL(H2, ADD(2.0, 1.0)), H1)

```

```

GP population- size: 20
Gen 01/10 | Best Fitness: 102.2 | Formula: ADD(MUL(MUL(1.0, H1), H2), H1)
Gen 02/10 | Best Fitness: 98.6 | Formula: MUL(H2, H1)
Gen 03/10 | Best Fitness: 71.8 | Formula: ADD(MUL(DistExit, H2), H1)
Gen 04/10 | Best Fitness: 71.8 | Formula: ADD(MUL(DistExit, H2), H1)
Gen 05/10 | Best Fitness: 71.8 | Formula: ADD(MUL(DistExit, H2), H1)
Gen 06/10 | Best Fitness: 70.4 | Formula: ADD(MUL(H2, H2), H2)
Gen 07/10 | Best Fitness: 70.4 | Formula: ADD(MUL(H2, H2), H2)
Gen 08/10 | Best Fitness: 70.4 | Formula: ADD(MUL(H2, H2), H2)
Gen 09/10 | Best Fitness: 70.4 | Formula: ADD(MUL(H2, H2), H2)
Gen 10/10 | Best Fitness: 70.4 | Formula: ADD(MUL(H2, H2), H2)
GEP population- size: 20, head length: 6
Gen 01/10 | Best Fitness: 91.8 | Formula: MUL(DistExit, MAX(SUB(SUB(H2, 1.0), H1), H2))
Gen 02/10 | Best Fitness: 91.8 | Formula: MUL(DistExit, MAX(SUB(SUB(H2, 1.0), H1), H2))
Gen 03/10 | Best Fitness: 91.8 | Formula: MUL(DistExit, MAX(SUB(SUB(H2, 1.0), H1), H2))
Gen 04/10 | Best Fitness: 91.8 | Formula: MUL(DistExit, MAX(SUB(SUB(H2, 1.0), H1), H2))
Gen 05/10 | Best Fitness: 81.8 | Formula: MUL(MAX(MUL(DistExit, 1.0), DistExit), H2)
Gen 06/10 | Best Fitness: 81.8 | Formula: MUL(MAX(MUL(DistExit, 1.0), DistExit), H2)
Gen 07/10 | Best Fitness: 61.8 | Formula: MUL(DistExit, H2)
Gen 08/10 | Best Fitness: 61.8 | Formula: MUL(DistExit, H2)
Gen 09/10 | Best Fitness: 61.8 | Formula: MUL(DistExit, H2)
Gen 10/10 | Best Fitness: 61.8 | Formula: MUL(DistExit, H2)

```

Method	Quality (avg N)	Diversity (distinct/2)	Gen. time (s)
GP	1201.0	2	47.0
GEP	1470.3	2	38.8

```

GP formulas across runs:
MUL(H2, H2)
ADD(MUL(H2, H2), H2)
GEP formulas across runs:
MUL(MUL(H2, ADD(2.0, 1.0)), H1)
MUL(DistExit, H2)

```

ההשוואה בין GP ל GEP בוצעה לפי שלוש המדדים הדרושים:

- א. איכות (מספר צמתים שפותחו): GP הראה עליונות באיכות הפתרון 1201 צמתים בממוצע לעומת 1470.3 ב GEP. ה GP השיג זאת מכיוון שהתכנס לנוסחה מנפחת אגרסיבית יותר, בעוד GEP התכנס ל $MUL(DistExit*H2)$ שתי הנוסחאות שייכות לאותה משפחה של $weighted A^*$.
 - ב. גיוון: שתי השיטות הציגו גיוון זהה בסדרת הריצות- 2 נוסחאות שונות מתוך 2 ריצות בשתייהן. האלגוריתם נותר לחקור בתוך אותה משפחת פונקציות המבוססת על הטיה של H2.
 - ג. זמן יצירה: במדד זה GEP מנצח והיה מהיר יותר בשלב האימון האבולוציוני, 38.8 לעומת 47 שניות של GP. וזה צפוי גם כי האופרטורים הגנטיים הפועלים ישירות על גנום לינארי רציף זולים בהרבה מבחינת חישובית מפעולות על מבנה נתונים של עץ.
- כך שעל פני 40 בעיות, היוריסטיקה המפותחת גם של GP וגם GEP פתרה בהצלחה את כל ה 40, כמו גם H2, אבל פתחה כ 37% (GP) פחות מצבים בממוצע, 1201 לעומת 1921.3 ב H2, וכן רצה גם כ 38% מהר יותר. בכך קבלנו שיפור משמעותי במדדים שלשמם נוצרה פונקציית הפיטנס.
- האלגוריתם A^* ממין את תור העדיפויות לפי $f=g+h$, כאשר h קבילה, הפונקציה f לא תעריך ביתר את העלות האמיתית. ולכן A^* מוכיח מציאת מסלול קצר ביותר, אבל מפתח מצבים רבים לשלילת חלופות.
 - הנוסחה החדשה של GP מכפילה את H2 בעצמו וב 2, דבר שגורם להערכת יתר לכל המצבים. מצבים עם כמות חוסמים גדולה, מקבלים ניפוח גדול עם העלאה של H2 בריבוע. ובכך A^* נדחף להמשיך בחמדנות לעבר היציאה במקום לחקור חלופות. התוצאה היא עץ חיפוש קטן משמעותית, במחיר של מסלול ארוך יותר.
 - כדי לנתח את המחיר של המהירות הזו, עקבתי גם אחרי מסלול הפתרון. וראינו כבר בתרגיל הקודם ש A^* בשילובה מוצא תמיד את המסלול האופטימלי. ומתוך 40 הפאזלים, היוריסטיקה של GP כמעט ולא מצאה פתרון אופטימלי מבחינת המסלול. וזהו הטריידאוף- היוריסטיקה הלא קבילה החדשה מכוונת את A^* בחמדנות לעבר המטרה, מפתחת הרבה פחות מצבים, אבל המסלולים לא אופטימליים.
 - מכיוון ש GP ו GEP הם סטוכסטים, הרצות שונות מתכנסות לנוסחאות שונות, עם זאת ההתכנסות היא תמיד לאותה משפחת פונקציות- יוריסטיקת H2 שמנפחת על ידי קבוע או גורמים אחרים משתנים. כך שיצירת יוריסטיקה בלתי קבילה בסגנון של $weighted A^*$ מראה כי הפיטנס כן מתגמל התנהגות כזו תמיד וזה לא מקרי.

7. מסקנות:

- א. GP ו GEP פיתחו יוריסטיקות טובות יותר מ H2 בהיבט הנדרש בתרגיל:
ירידה של כ 37% בכמות הצמתים שפותחו וירידה של כ38% בזמן הריצה,
תוך כך שהוא פותר 40/40 חידות. וכן גם GEP הציג שיפור גדול מול H2 עם
1470.3 צמתים בממוצע.
- ב. גילוי לבד של $weighted A^*$ בשתי השיטות, האבולוציה בהן גילתה לבד את
העיקרון של H2, ובכך יישם את מה שלמדנו בקורס לטריידאוף בין
אופטימליות למהירות. שתי שיטות עם ייצוג גנומי שונה התכנסו לאותו עקרון
וזוה מראה כיוון זה אכן מתוגמל.
- ג. ההאצה בזמני החיפוש גבתה מחיר בטיב אורך הפתרונות, בדיוק כפי שראינו
בתאוריה על יוריסטיקות לא קבילות.
- ד. פונקציית הפיטנס מנעה התנפחות: קנס הסיבוכיות הפועל על גודל העץ,
הבטיח שהנוסחאות שפותחו יהיו מהירות וקומפקטיות, כמו שבאחת הריצות
שלי נמצאה הפונקציה $MUL(2.0, H2)$ שהיא ככל הנראה התמציתית ביותר
שיכלה להמצא ליוריסטיקה.
- ה. GP vs GEP: GP יצר פתרון איכותי יותר לו חיסכון בצמתים מפותחים, אך
אלגוריתם GEP היה מהיר ויעיל יותר ביצירת ההיוריסטיקות, אימון, זאת
בזכות הפשטות של הייצוג הלינארי.

• מסקנות כוללות:

חלק א':

בצעתי השוואה בין שבעה אלגוריתמים לפתרון בעיית CVRP, וראיתי מהתוצאות כי אין מנצח מוחלט בין כל מופעי הבעיות: GA island ניצח במופעים הקטנים, ומנגד ALNS בבינוניים ובגדולים, ו ACO הביא את התוצאה הטובה ביותר ל X-n101 הקשה (למרות שלא היה חוקי). כל אלגוריתם טוב בסביבה אחרת והבחירה תלויה בגודל המופע ובמבנה שלו. יישמתי גם את עקרון החוקיות נגד אופטימליות שלמדנו בהרצאה, זאת על ידי החלפת אילוצי הקיבולת ב strategic oscillation, ובכך אפשר הגעה למרחב הפתרונות הלא חוקיים כדי להמנע מקיפאון.

חלק ב':

במקום לבחור, GP ו GEP פיתחו את היוריסטיקה של A^* . האבולוציה גלתה את העיקרון של $weighted A^*$ בו הטריידאוף בין אופטימליות למהירות. כל זה התכנס לנוסחה קומפקטית שמפתחת בממוצע כ 37% פחות מצבים לעומת היוריסטיקה H2 מהתרגיל הראשון. אך כמובן המחיר שלה היה בפתרון ארוך יותר ברוב המקרים. השוואת השיטות הראתה כי GEP מספק יתרון בזמני הפיתוח בשל הייצוג הלינארי, בעוד GP מוביל באיכות הסופית.

כולל:

בשני החלקים, איכות היוריסטיקה היא זו שהכריעה. בחלק א' בחרתי ובחנתי את היוריסטיקות ידניות, והטמעתי אותן באלגוריתמי החיפוש. בחלק ב' GP אפשר לפתח היוריסטיקה בעצמו. כך שבשני המקרים, מה שקבע את התוצאה הסופית היה הבנת מבנה הבעיה והטריידאופים- חוקיות מול אופטימליות בחלק א' ומהירות מול אופטימליות המסלול בחלק ב'.

• שיפורים לעתיד:

- שילוב אופרטורים נוספים, כי הסתמכות רק על אופרטור העברה גורמת לחיפוש המקומי להתקע כאשר הטיפול הוא במופעים הצפופים, כמו במופע הקשה שדנתי בו. שילוב של אופרטור Swap למשל, יאפשר החלפה של לקוחות בין משאיות עמוסות, וכך לדלג על מינימום מקומי בלי לחרוג מהקיבולת.
- הרחבת קבוצת הפונקציות של GP, כמו גם הוספת חילוק או תנאים לוגיים, זאת כדי להשוות בין יותר סוגי היוריסטיקות.
- הרצת GP עם פונקציית פיטנס שמשלבת גם משקל על אורך מסלול, כדי למצוא גם פתרונות אופטימליים רצים מהר וביעילות יותר.

- שימוש בLLMs:

שימוש בgeminii לייצור טבלאות הפלט וההשוואות בצורה מסודרת וקריאה, עזרה ביצירת plots, עזרה קלה בטיפול בבאגים, וכן כיוון בהתאמת המטא יוריסטיקות למרחב רציף עם ACKLEY.

החומר התאורטי שנעזרתי בו היה חומר ההרצאות, וכן ב ACOR (Socha & Dorigo 2008), בו נעזרתי להתאמת ACO למרחב רציף לACKLEY.

- רשימת הקבצים בהגשה:

- CVRP.py - קוד מקור לחלק הראשון
- RH_GP.py - קוד מקור לחלק השני
- CVRP.exe - קובץ EXE לחלק הראשון
- RH_GP.exe - קובץ EXE לחלק השני
- rh.txt - קובץ החידות של rush hour
- 6 קבצי בעיות CVRP (.vrp)
- Report.pdf - קובץ הדוח הנוכחי
- Readme.txt - הוראות הרצה